Image Segmentation and Specification Retrieval of Microcontrollers

Report submitted to

Indian Institute of Technology, Kharagpur

in partial fulfilment for the award of the degree

of

Master of Technology in Vision and Intelligent Systems

by

Subhadeep Mondal (24EC65R30)

Under the Supervision of

Dr. Indrajit Chakrabarti

&

Dr. Saumik Bhattacharya

Department Of Electronics And Electrical Communication Engineering ,INDIAN INSTITUTE OF TECHNOLOGY, KHARAGPUR

Nov 2025

DECLARATION

I certify that

a. The work contained in this report is original and has been done by me under the guidance of my supervisor(s).
b. The work has not been submitted to any other Institute for any degree or diploma.
c. I have followed the guidelines provided by the Institute in preparing the report.
d. I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.
e. Whenever I have used materials (data, theoretical analysis, figures, and text) from other sources, I have given due credit to them by citing them in the text of the report and giving their details in the references. Further, I have taken permission from the copyright owners of the sources whenever necessary.

Signature of Student

Department of Electronics & Electrical Communication Engineering

Indian Institute of Technology, Kharagpur, India-721302

CERTIFICATE

This is to certify that the Dissertation Report entitled "**Image Segmentation and Specification Retrieval of Microcontrollers**" submitted by **Mr. Subhadeep Mondal(24EC65R30)** to the Indian Institute of Technology, Kharagpur, is a record of Bonafide project work carried out by him under my supervision and guidance and is worthy of consideration as a partial fulfilment for the award of the degree of Master of Technology in Vision and Intelligent Systems of the Institute.

Place: Kharagpur

Supervisor

Date:

Department of Electronics and Electrical Communication Engineering

Indian Institute of Technology, Kharagpur

ACKNOWLEDGEMENTS

This project is a culmination of the efforts and contributions of several individuals who have provided their support, guidance, and inspiration throughout the span of this project. I am very grateful to all those who have extended their valuable time and assistance to complete this phase of project.

I would like to express my heartfelt gratitude to my supervisors, **Dr. Indrajit Chakrabarti & Dr. Saumik Bhattacharya** Department of Electronics and Electrical Communication Engineering, India Institute of Technology, Kharagpur, for their invaluable guidance and continuous encouragement during this phase of project.

Lastly, I would like to thank God for keeping me in good health throughout the duration of the project. Also, Thanks to my parents, who always inspire and guide me to scale new heights.

- Subhadeep Mondal

Abstract

This report presents **MCUDetector**, a custom ultra-lightweight object detection system designed specifically for identifying 14 classes of microcontroller boards including Arduino, Raspberry Pi, ESP32, NodeMCU, ARM Cortex variants and the 8051 family. The work ranges from architecture design through training, quantization and mobile deployment, with the goal of achieving accuracy competitive with mainstream YOLO detectors at a fraction of their parameter count.

The core contribution is a novel detection architecture built around a **RepViT-CSP hybrid backbone** paired with a **BiFPN neck** featuring **SimAM parameter-free attention** at the fusion nodes and lightweight **depthwise-separable decoupled detection heads**. The primary model variant, uses only **0.377M parameters** — roughly 8.5 times smaller than YOLOv8-nano yet achieves **99.43% mAP@0.5** and **93.07% mAP@0.5:0.95** on the custom MCU dataset at 320×320 resolution.

An alternative backbone using **StarNet blocks** from the "Rewrite the Stars" paper (CVPR 2024) was explored as an ablation, producing the variant at 0.342M parameters with fully INT8-safe HardSwish activations throughout. A complete **post-training quantization pipeline** (PyTorch → ONNX → TFLite) with post-training quantization achieves **3.01× model compression** to 0.51 MB in INT8 with essentially zero accuracy loss. The system is deployed on a **ARM Cpu Integrated** smartphone through a Flutter-based mobile application, enabling real-time capture, detection and datasheet retrieval.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

The rapid growth of the Internet of Things and embedded systems has led to rapid increase of microcontroller boards on printed circuit boards (PCBs) across consumer electronics, industrial automation and educational laboratories. Identifying these boards knowing what chip family they belong to, what their pin configurations look like, or where to find their datasheets remains a surprisingly manual and time-consuming process, especially for students and technicians working with unfamiliar hardware.

In my third semester I built an end-to-end pipeline [1] that used a pretrained YOLOv8s detector to localise microcontrollers on PCB images, a custom CRNN-based OCR module to read their part-number markings and a FastAPI+MongoDB backend to retrieve datasheets. That system worked well as a prototype, but it relied entirely on off-the-shelf detection models with millions of parameters, making it impractical for on-device deployment on a mobile phone without server round-trips.

This semester the project took a fundamentally different direction. Instead of using a standard YOLO model, I designed and trained **MCUDetector** from scratch a purpose-built detection architecture that is small enough to run on a mid-range smartphone yet accurate enough to outperform the pretrained YOLOv8 baselines on our dataset. The key contributions of this work are:

1. **A novel lightweight architecture** combining RepViT-style re-parameterisable depthwise convolutions with Cross Stage Partial (CSP) feature aggregation, a BiFPN neck with SimAM attention and depthwise-separable decoupled detection heads all in 0.373M parameters.

2. **A StarNet-based ablation variant** that replaces RepViT blocks with star-operation blocks from CVPR 2024, demonstrating that element-wise feature multiplication provides competitive accuracy at even fewer parameters (0.342M).

3. **An end-to-end quantization pipeline** that converts PyTorch models through ONNX to TFLite with FP32, FP16 and INT8 quantization, achieving 3.01× compression with

no measurable accuracy degradation.

4. **A Flutter-based mobile application** deployed on a ARM mobile device that performs capture, on-device detection and datasheet retrieval in a single user-facing interface.

5. **Comprehensive ablation studies** covering resolution scaling (320 vs. 384 vs. 512), backbone variants, attention mechanisms and loss function components.

The rest of this report is organised as follows. Chapter 2 reviews the relevant literature. Chapter 3 describes the proposed MCUDetector architecture in detail. Chapter 4 covers the training methodology and dataset. Chapter 5 presents the quantization pipeline. Chapter 6 describes the Flutter mobile application. Chapter 7 presents the experimental results. Chapter 8 concludes the report and discusses future work.

Chapter 2

Literature Review

2.1 Evolution of Real-Time Object Detection

Object detection has undergone a major transformation over the last decade. The R-CNN family [7, 8, 9] introduced the idea of region proposals followed by CNN classification,though highly accurate but these two-stage pipelines remained too slow for real-time use. The YOLO family [2, 3, 4, 5, 6] changed the game by framing detection as a single-pass regression problem, predicting bounding boxes and class probabilities simultaneously from a single forward pass through the network.

YOLOv8 [6] represents the current state of the art in the YOLO lineage, introducing anchor-free detection heads, C2f blocks for better gradient flow and a decoupled classification-regression head design. However, even YOLOv8-nano has approximately 3.2 million parameters, which is far more than necessary for a domain-specific task with only 14 classes.

2.2 Lightweight Architecture Design

The MobileNet family [10, 11] popularised depthwise separable convolutions as a way to drastically cut computation costs. RepVGG [12] and later RepViT [13] showed that multi-branch training architectures could be collapsed into single-path inference branch through structural re-parameterisation, giving the accuracy benefits of complex training topologies without the runtime cost.

The Cross Stage Partial (CSP) connection pattern [14] splits feature channels into two groups — one that passes through a sequence of bottleneck blocks and one that bypasses them then fuses the results. This design reduces redundant gradient information and has been widely adopted in modern detectors including YOLOv5 and YOLOv8.

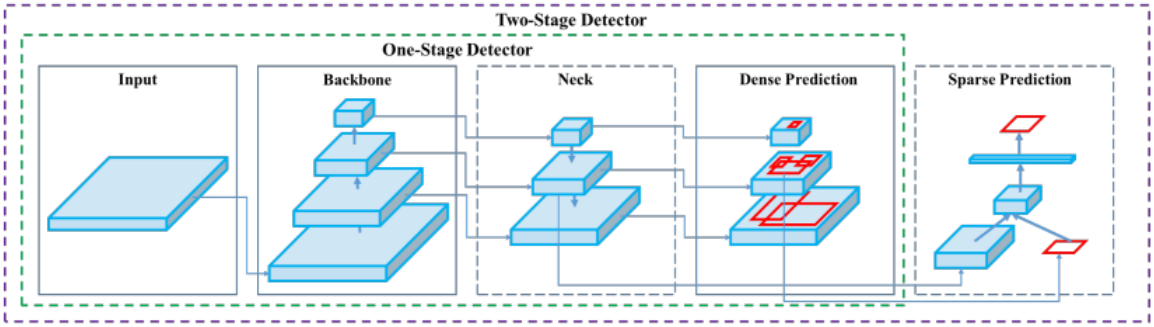


Figure 2.1: General object detection pipeline illustrating the backbone for feature extraction, neck for multi-scale feature aggregation (e.g., FPN/BiFPN), and detection head for dense or sparse predictions. Adapted from YOLOv4 [5].

2.3 Attention Mechanisms

Channel attention (Squeeze and Excitation) (SE-Net [15]) and spatial attention (CBAM [16]) add learnable parameters to re-weight feature maps. SimAM [17] takes a different approach by deriving attention weights from the energy function of a neuron without any learnable parameters, making it fully compatible with INT8 quantization and adding zero overhead to the model size.

2.4 Feature Pyramid Networks

Feature Pyramid Networks (FPN) [18] established the standard top-down pathway for multi-scale feature fusion. BiFPN [19] extended this with bidirectional connections and learnable weighted fusion, allowing the network to decide how much weight to give each input feature scale at each fusion node.

2.5 StarNet: Rewrite the Stars

Ma *et al.* [20] proposed the star operation element-wise multiplication of two linear projections of the same feature map as a way to implicitly map features into a very high-dimensional polynomial space without explicit high-rank transformations. Their StarNet architecture demonstrated that this simple operation could match or exceed the accuracy of much more complex attention mechanisms, making it attractive for lightweight detector backbones.

2.6 Post-Training Quantization

Post-training quantization (PTQ) converts floating-point weights and activations to lower-precision formats (FP16, INT8) without retraining [23]. TensorFlow Lite [24] provides a mature toolchain for converting ONNX or TensorFlow models to quantized TFLite format suitable for mobile deployment. Outlier-Aware Quantization (OAQ) [25] handles the challenge of weight outliers that can degrade INT8 accuracy by using per-channel quantization for weights and representative calibration data.

2.7 Gaps Addressed by This Work

The existing literature offers powerful building blocks, but no prior work has combined RepViT re-parameterisation, CSP aggregation, SimAM attention in BiFPN fusion nodes and star-operation ablations into a single sub-400K parameter detector specifically targeting microcontroller board identification with an end-to-end quantization and mobile deployment pipeline. This project aims to fill that gap.

Chapter 3

Proposed MCUDetector Architecture

This chapter describes the MCUDetector architecture in detail. Two variants were developed: the primary (RepViT-CSP backbone) and the ablation (StarNet-CSP backbone). Both share the same BiFPN neck and decoupled detection head; they differ only in the backbone blocks.

3.1 Design Philosophy

The guiding principle behind MCUDetector is *right-sized design*: every component is scaled to the complexity of the actual task (14 classes, ∼6 000 training images(inflate)) rather than borrowed wholesale from general-purpose detectors designed for 80+ COCO classes. The specific design choices that follow from this principle include:

- Two-scale detection (P3 at stride 4, P4 at stride 8) instead of the three-scale design used by YOLOv8n. Microcontroller boards occupy a relatively narrow size range in our images, so a third scale would add parameters without meaningful accuracy gain.
- Channel widths of 48–96–192 instead of the 64–128–256 used by YOLOv8-nano. For 14 classes, the representational capacity of 96 FPN channels is more than sufficient.
- Single CSP block per backbone stage instead of the double blocks common in larger detectors. With our dataset size, the second block provides diminishing returns.
- Depthwise-separable convolutions in the detection head branches, replacing the full convolutions used by YOLOX-style heads.

3.2 RepViT-CSP Backbone (V2-Lite)

The backbone produces two feature maps: P3 at stride 4 with 96 channels and P4 at stride 8 with 192 channels.

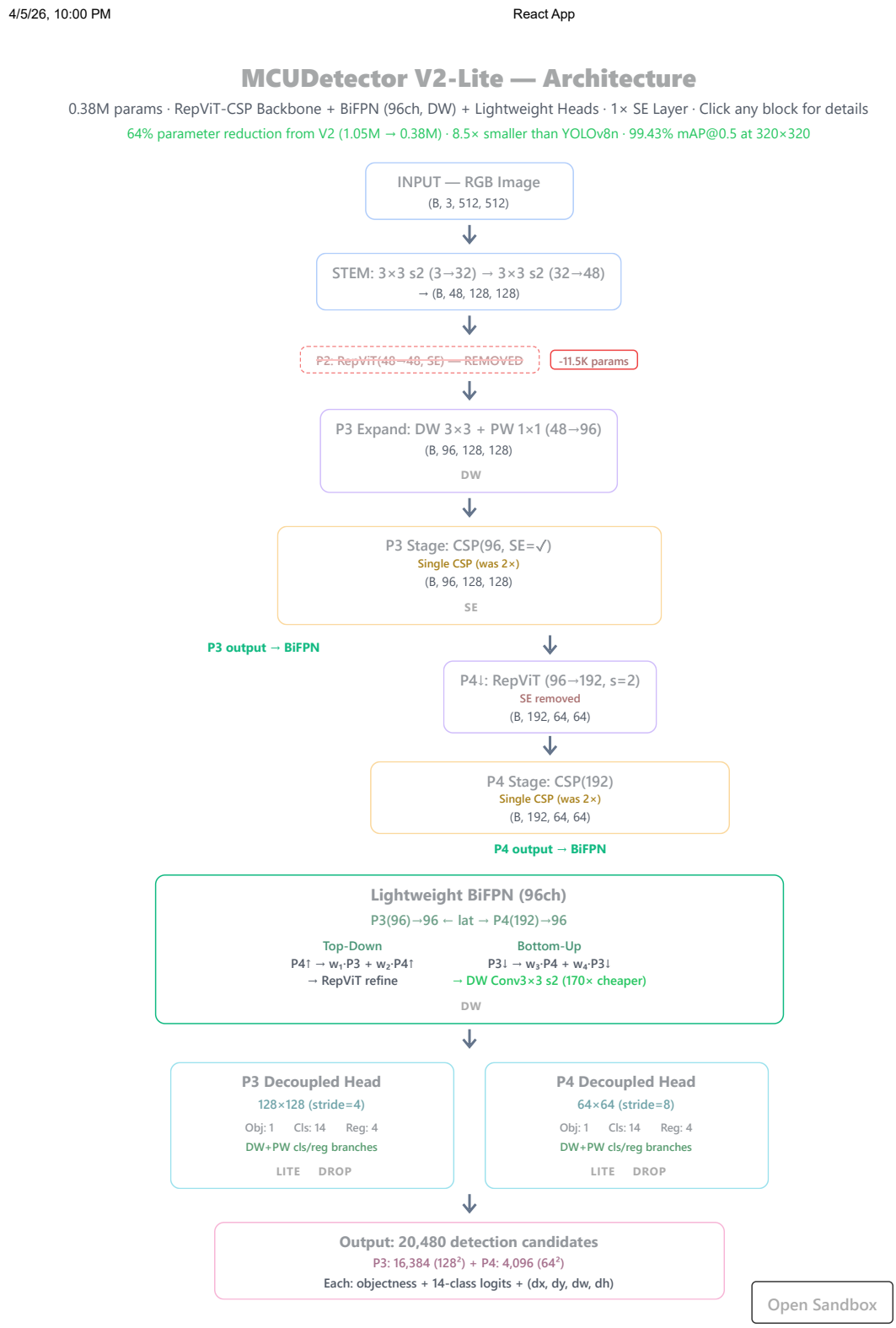


Figure 3.1: RepViT Backbone Architecture used in MCUDetector

3.2.1 Stem (Common in both RepViT and Starnet)

The stem consists of two 3×3 convolutions with stride 2 each, expanding from 3 input channels (RGB) to 32 and then to 48 channels, with BatchNorm and HardSwish activation after each convolution. This gives a $4\times$ spatial downsampling from the input resolution.

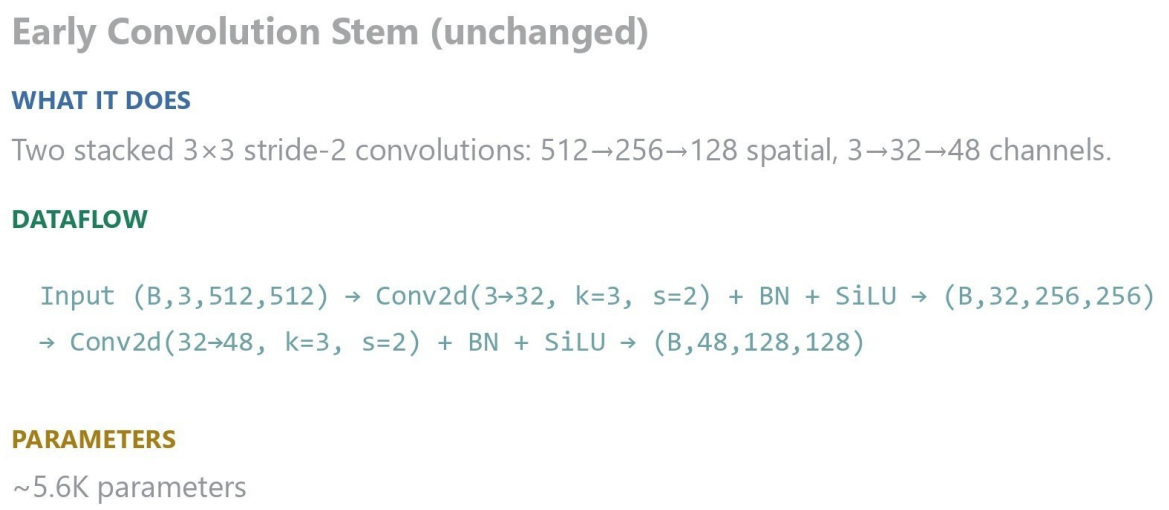


Figure 3.2: Interactive visualization of the MCUDetector Stem

3.2.2 RepViT Blocks

Each RepViT block follows a token-mixer + channel-mixer design inspired by [13]. The token mixer is a **re-parameterisable depthwise convolution** (RepDWConvSR) that maintains three parallel branches during training: a 3×3 depthwise conv, a 1×1 depthwise conv and an identity-with-BatchNorm branch. At inference time, these branches are algebraically fused into a single 3×3 depthwise convolution, eliminating all multi-branch overhead:

$$\mathbf{W}_{\text{fused}} = \mathbf{W}_{3\times3} + \text{pad}(\mathbf{W}_{1\times1}) + \text{pad}(\mathbf{W}_{\text{id}}) \tag{3.1}$$

The channel mixer is a two-layer MLP (pointwise conv $\rightarrow$ BN $\rightarrow$ HardSwish $\rightarrow$ pointwise conv $\rightarrow$ BN) with an expansion ratio of 2. Residual connections with a scaling factor of 0.5 stabilise training. An optional Squeeze-and-Excitation (SE) module provides channel attention where beneficial.

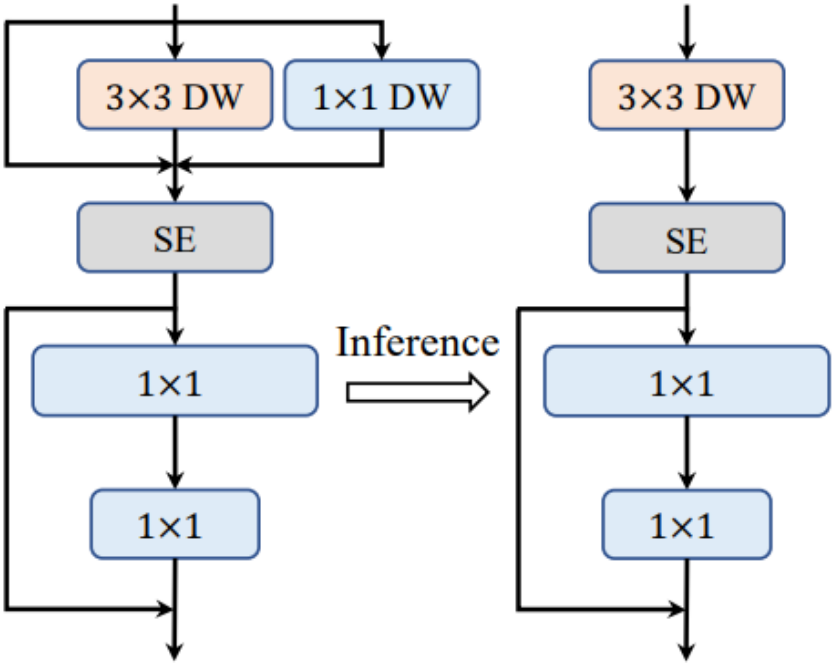


Figure 3.3: RepViT block architecture illustrating the separation of token mixer (depthwise convolution) and channel mixer (pointwise MLP), along with structural re-parameterization for efficient inference. Adapted from [13].

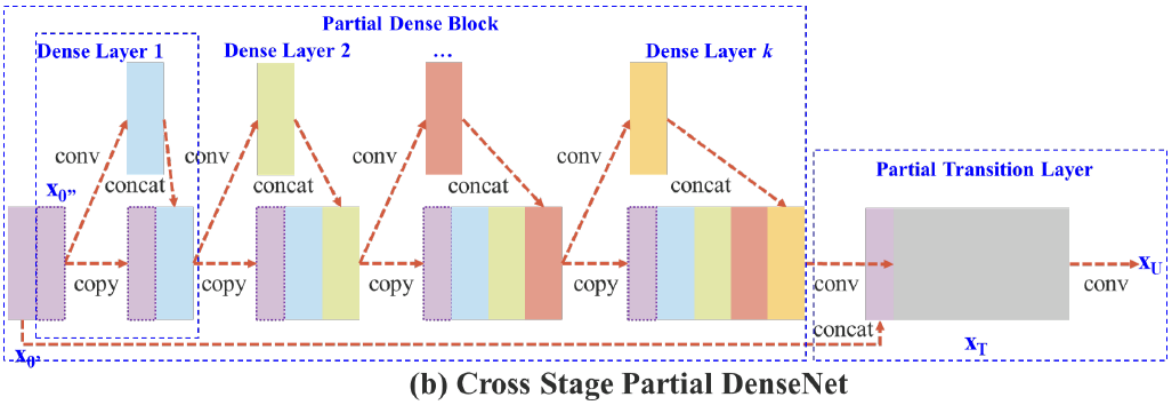
3.2.3 CSP Aggregation

The Cross Stage Partial (CSP) aggregation block splits the input feature channels into two partitions: a bypass branch and a transformation branch. The transformation branch passes through a sequence of feature extraction blocks, while the bypass branch preserves the original features. The two branches are then concatenated and fused using a 1×1 convolution.

In this work, the transformation branch can be instantiated using different backbone blocks such as RepViT or StarNet. This design makes the CSP module flexible and backbone-agnostic, enabling efficient feature reuse while reducing redundant gradient information.

Formally, let $\mathcal{F}(\cdot)$ denote the feature transformation block (RepViT or StarNet). Then,

$$\mathbf{Y} = \text{Conv}_{1\times1}\left(\text{Concat}(\mathbf{X}_{\text{bypass}}, \mathcal{F}(\mathbf{X}_{\text{transform}}))\right)$$



(b) Cross Stage Partial DenseNet

Figure 3.4: Cross Stage Partial (CSP) architecture illustrating channel splitting, partial computation, and feature fusion. A portion of the feature maps bypasses computational blocks while the remaining channels undergo transformation before concatenation. Adapted from [14].

3.2.4 Backbone Stage Design

The backbone is organized into hierarchical stages producing feature maps at multiple resolutions. Depthwise separable convolutions are used to reduce computational complexity in the proposed architecture. A depthwise convolution performs spatial filtering independently on each channel, followed by a pointwise 1×1 convolution for channel mixing. This factorization significantly reduces parameters and computation compared to standard convolution.

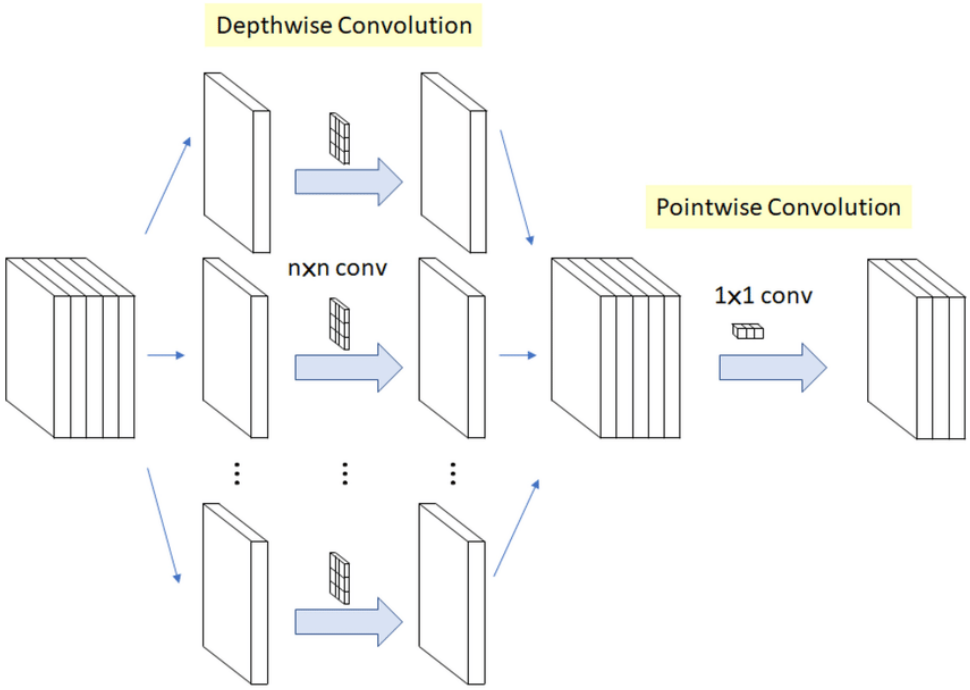


Figure 3.5: Depthwise separable convolution: depthwise (spatial filtering) followed by pointwise (channel mixing)

P3 Stage

The P3 stage operates at stride 4 resolution. Instead of using a full RepViT block, a lightweight depthwise separable convolution is used for channel expansion from 48 to 96 channels.

- Depthwise 3×3 convolution (spatial mixing)
- Pointwise 1×1 convolution (channel expansion)

This is followed by a CSP aggregation block.

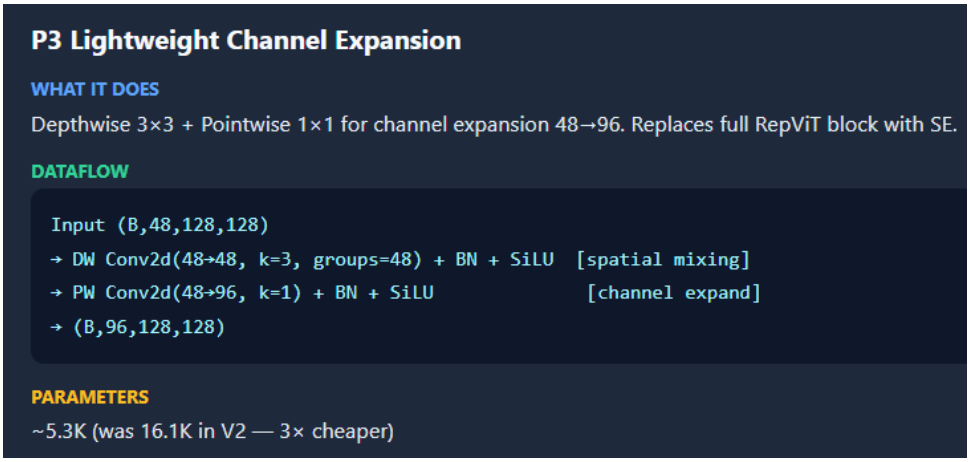


Figure 3.6: Lightweight P3 channel expansion using depthwise separable convolution

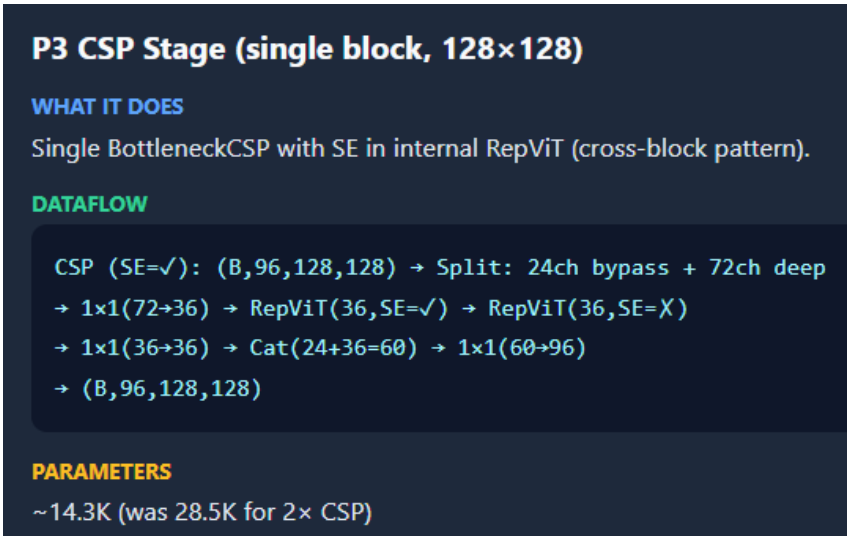


Figure 3.7: CSP aggregation in P3 stage

P4 Stage

The P4 stage performs spatial downsampling using a RepViT block with stride 2, increasing channels from 96 to 192.

This is followed by a CSP aggregation block to enhance feature reuse and reduce redundancy.

Figure 3.8: P4 downsampling using RepViT block (stride 2)

Figure 3.9: CSP aggregation in P4 stage

3.2.5 Stage Layout

The backbone stages are:

Table 3.1: MCUDetector V2-Lite backbone stage configuration.

Stage	Module	Channels	Stride	Params
Stem	$2\times$ Conv 3×3	$3 \to 32 \to 48$	$4\times$	15 264
P3 Expand	DW 3×3 + PW 1×1	$48 \to 96$	$1\times$	5 328
P3 CSP	BottleneckCSP (2 RepViT)	96	$1\times$	30 768
P4 Down	RepViT (stride 2)	$96 \to 192$	$2\times$	43 008
P4 CSP	BottleneckCSP (2 RepViT)	192	$1\times$	107 712

3.3 StarNet-CSP Backbone

The Starnet model variant replaces the RepViT blocks inside each CSP with StarBlocks [20]. Each StarBlock applies a depthwise 3×3 convolution followed by two parallel pointwise projections f_1 and f_2, with the outputs combined via element-wise multiplication (the "star operation"):

$$\mathbf{y} = \mathrm{ReLU6}(f_1(\mathbf{x})) \odot f_2(\mathbf{x}) \tag{3.2}$$

This multiplication implicitly maps features into a polynomial space of dimension d^{2^L} where L is the number of stacked star layers and d is the channel count. This gives the network enormous implicit capacity, which we found caused overfitting on our relatively small dataset. We mitigated this with:

- Dropout2d(0.1) applied to the star product output.
- Residual scaling of 0.5 (i.e. $\mathbf{out} = 0.5 \cdot \mathbf{y} + \mathbf{x}_{\text{residual}}$).
- Reduced CSP blocks at the P3 stage.

All activations in Starnet Model use HardSwish (via the aliased `SiLU` wrapper in `utils.py`), making the entire model INT8-safe. The Starnet model has 0.348M parameters.

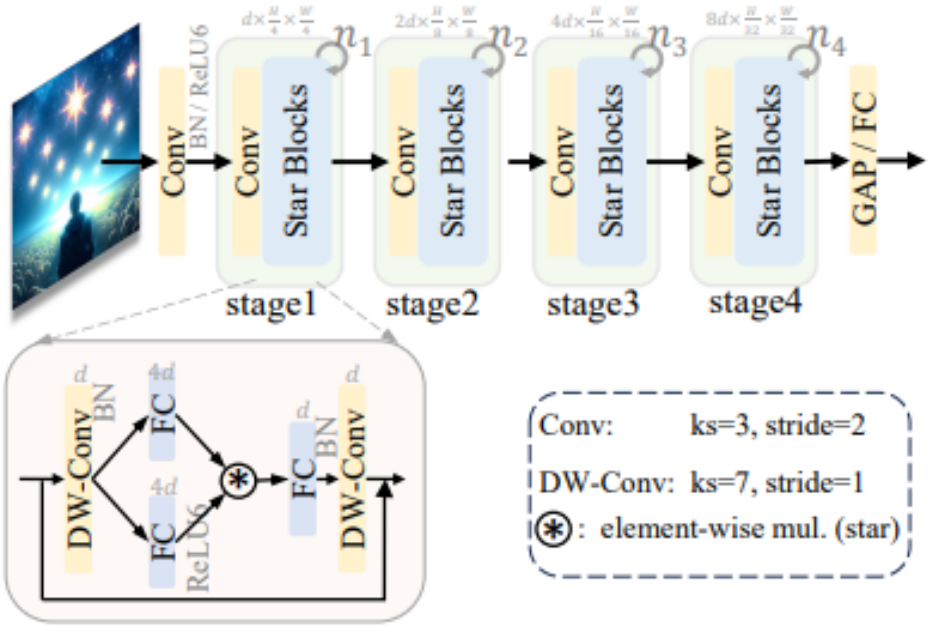


Figure 3.10: StarNet block architecture illustrating dual-branch pointwise projections and element-wise multiplication (star operation). Adapted from Ma *et al.* [20].

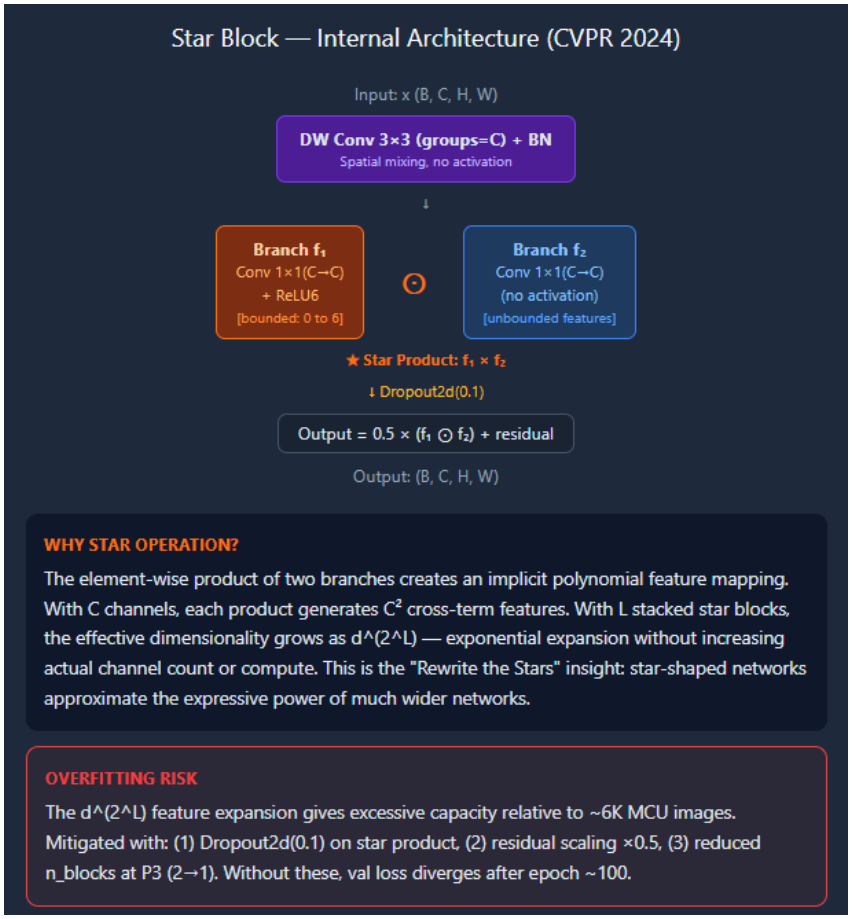


Figure 3.11: Starnet block in detail. Adapted from Ma *et al.* [20].

3.4 BiFPN Neck with SimAM Attention

The neck fuses P3 and P4 features using a single BiFPN module with learnable weighted fusion:

$$\mathbf{P3}_{\text{td}} = \text{RepViT}\left(\frac{w_1 \cdot \mathbf{P3}_{\text{lat}} + w_2 \cdot \text{Up}(\mathbf{P4}_{\text{lat}})}{w_1 + w_2 + \epsilon}\right) \tag{3.3}$$

$$\mathbf{P4}_{\text{out}} = \text{RepViT}\left(\frac{w_3 \cdot \mathbf{P4}_{\text{lat}} + w_4 \cdot \text{DW-Down}(\mathbf{P3}_{\text{td}})}{w_3 + w_4 + \epsilon}\right) \tag{3.4}$$

The downsampling in the bottom-up path uses a **depthwise** 3×3 **convolution** with stride 2 instead of the standard convolution used in the original BiFPN. This single change saves approximately 147 000 parameters (the old standard conv was the single largest layer in the model at 14% of total parameters).

In the StarNet and RepViT variant, **SimAM** attention modules are placed at the P3 and P4 fusion nodes inside the BiFPN. SimAM requires zero learnable parameters and derives spatial-channel attention weights from a 3D energy function:

$$e_t^* = \frac{4(\hat{\sigma}^2 + \lambda)}{(x_t - \hat{\mu})^2 + 2\hat{\sigma}^2 + 2\lambda} \tag{3.5}$$

where $\hat{\mu}$ and $\hat{\sigma}^2$ are the spatial mean and variance of the feature map and λ is a small constant. The output is $\mathbf{x} \cdot \sigma(1/e_t^*)$.

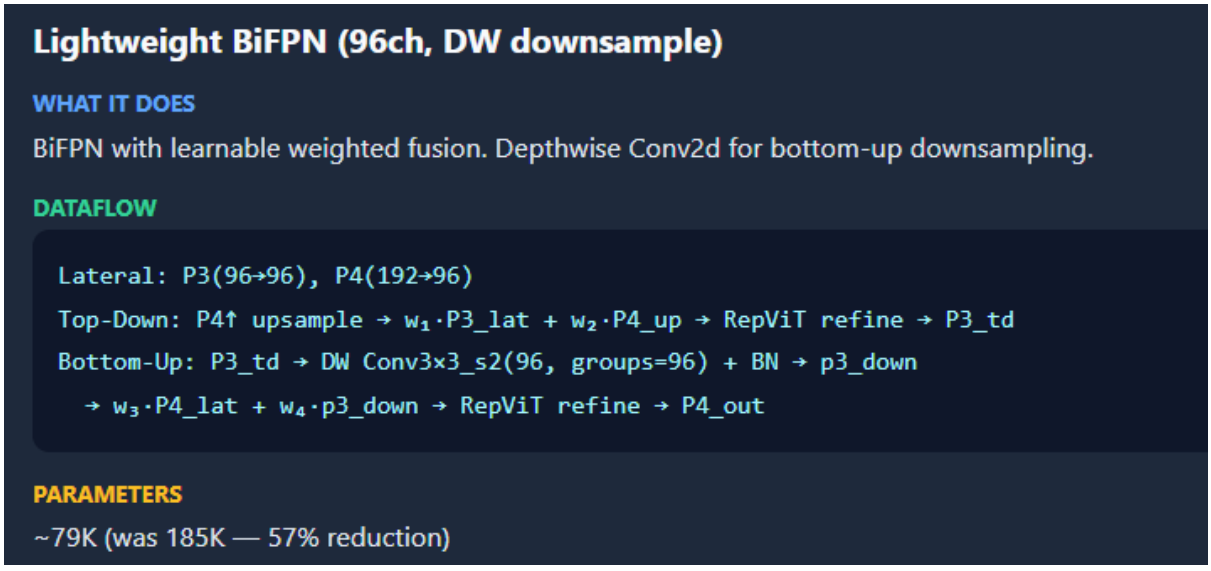


Figure 3.12: Lightweight BiFPN architecture with depthwise downsampling and learnable weighted feature fusion.

3.5 Decoupled Detection Head

Following the YOLOX design [22], the detection head decouples classification and regression into separate branches. Each per-scale head consists of:

1. A shared **RepViT refinement block** (96 channels).
2. A **classification branch**: DW $3 \times 3 \rightarrow$ BN $\rightarrow$ HardSwish $\rightarrow$ Dropout2d(0.1) $\rightarrow$ PW 1×1 projecting to $N_{\text{anchors}} \times N_{\text{classes}}$ outputs.
3. A **regression branch**: DW $3 \times 3 \rightarrow$ BN $\rightarrow$ HardSwish $\rightarrow$ PW 1×1 projecting to $N_{\text{anchors}} \times 4$ outputs.
4. An **objectness head**: a single 1×1 convolution projecting to $N_{\text{anchors}} \times 1$.

The classification head is initialised with a prior probability bias of 0.01 to address the foreground-background imbalance during early training.

Figure 3.13: Decoupled detection head with lightweight depthwise separable branches for classification and regression.

3.6 Loss Functions

The overall training objective is a weighted combination of bounding box regression, classification, and objectness losses:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{obj}}\mathcal{L}_{\text{obj}} \tag{3.6}$$

where λ_{box}, λ_{cls}, and λ_{obj} control the relative importance of each component.

3.6.1 Bounding Box Regression Loss

Bounding box regression is optimized using the Complete Intersection over Union (CIoU) loss, which considers overlap area, centre distance, and aspect ratio consistency:

$$\mathcal{L}_{\text{box}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v \tag{3.7}$$

where:

- IoU is the intersection-over-union between predicted and ground truth boxes.
- $\rho(\mathbf{b}, \mathbf{b}^{gt})$ is the Euclidean distance between box centres.
- c is the diagonal length of the smallest enclosing box.
- v measures the consistency of aspect ratios:

$$v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2 \tag{3.8}$$

$$\alpha = \frac{v}{(1 - \text{IoU}) + v} \tag{3.9}$$

This formulation improves localization accuracy by jointly optimizing spatial alignment and shape consistency.

3.6.2 Classification Loss

Classification is handled using Focal Loss [21], which addresses severe class imbalance by down-weighting easy negatives:

$$\mathcal{L}_{\text{cls}} = -\alpha(1 - p_t)^{\gamma}\log(p_t) \tag{3.10}$$

where:

- p_t is the predicted probability of the ground truth class,
- $\alpha = 0.25$ balances positive and negative samples,
- $\gamma = 2.0$ focuses learning on hard examples.

Focal loss is particularly effective in dense detection settings where background samples dominate.

3.6.3 Objectness Loss

Objectness prediction estimates whether an object exists in a given grid cell. It is optimized using binary cross-entropy with logits:

$$\mathcal{L}_{\text{obj}} = -\left[y \log(\sigma(o)) + (1-y)\log(1-\sigma(o))\right] \tag{3.11}$$

where:

- o is the predicted objectness logit,
- $\sigma(\cdot)$ is the sigmoid function,
- $y \in \{0, 1\}$ indicates object presence.

Unlike conventional normalization by positive samples, the loss is normalized by the total number of grid cells, ensuring stable gradients even in sparse detection scenarios.

3.6.4 Target Assignment: SimOTA-Lite

Target assignment is performed using a simplified SimOTA (Optimal Transport Assignment) strategy. For each ground truth box, the top-k nearest grid cells to its center are selected based on spatial proximity.

Let $\mathbf{g}$ denote the ground truth center and $\mathbf{c}_i$ denote candidate grid centers. The selection is based on:

$$d_i = \|\mathbf{c}_i - \mathbf{g}\|_2 \tag{3.12}$$

The k grid cells with the smallest distances are assigned as positive samples. In cases where multiple ground truths compete for the same cell, a conflict resolution strategy is applied based on minimum distance or highest IoU.

This lightweight assignment strategy reduces computational overhead compared to full optimal transport while retaining effective positive sample selection.

3.7 Parameter Comparison

Table 3.2: Parameter count comparison across model variants at 320×320.

Model	Parameters	GFLOPs	mAP@0.5 (%)	mAP@0.5:0.95 (%)
YOLOv8-nano	3.2M	8.7	–	–
YOLOv10-nano	2.7M	–	–	–
YOLOv11-nano	2.6M	–	–	–
MCUDetector RepViT	0.373M	2.46	99.43	93.07
MCUDetector StarNet	0.342M	2.29	99.49	95.75

Chapter 4

Training Methodology

4.1 Dataset

The dataset consists of approximately 6 000 annotated images spanning 14 classes of microcontroller and development boards. The per-class distribution in the training split ranges from 411 to 456 images per class, indicating a well-balanced dataset. The 14 classes are:

1. Arduino Due
2. Arduino Leonardo
3. Arduino Mega 2560 (Black and Yellow)
4. Arduino Mega 2560 (Black)
5. Arduino Mega 2560 (Blue)
6. Arduino Uno (Black)
7. Arduino Uno (Green)
8. Arduino Uno Camera Shield
9. Arduino Uno R3
10. Arduino Uno WiFi Shield
11. BeagleBone Black
12. Raspberry Pi 1 B+
13. Raspberry Pi 3 B+
14. Raspberry Pi A+

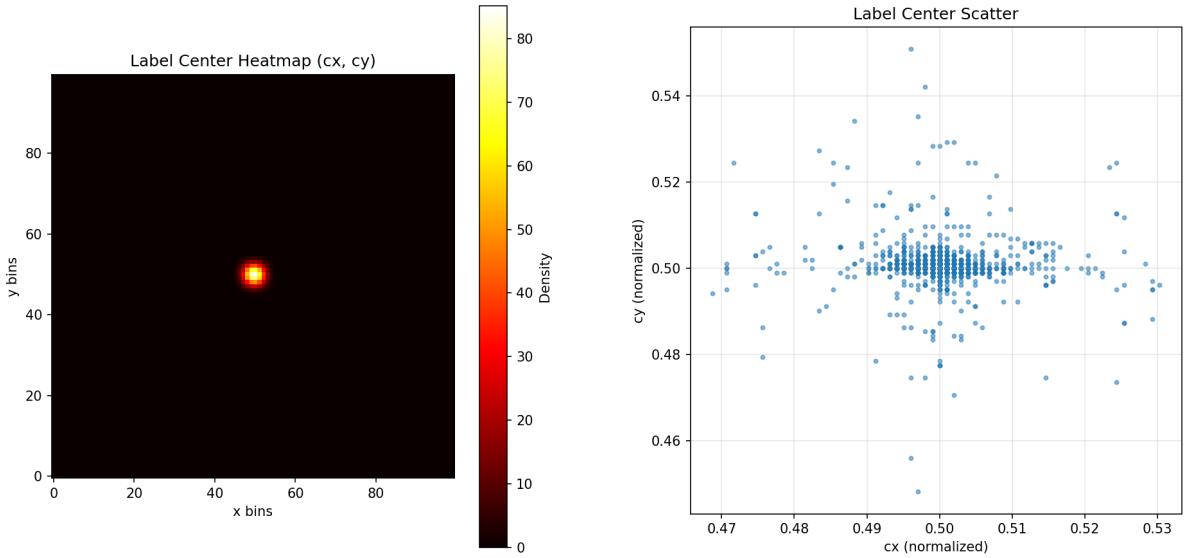
Images were collected from multiple sources: photographs taken under varying lighting conditions and angles, as well as images sourced from Roboflow Universe with manual verification. Each image was annotated in YOLO format (normalised c_x, c_y, w, h) with one bounding box per microcontroller instance. A separate test set of 883 images was held out for final evaluation and is never seen during training.

(a) Arduino Mega 2560 (b) ESP32 DevKit (c) NodeMCU ESP8266

Figure 4.1: Sample images from the MCU detection dataset showing diverse board types.

4.1.1 Dataset Distribution

The bounding box position and size distribution of the dataset is visualised in Figure 4.2. The heatmap shows that most bounding boxes are centred in the image, which is expected given the framing of individual boards. The scatter plot reveals a range of aspect ratios and sizes, confirming diversity in the captured viewpoints.



(a) Label centre heatmap (b) Label width-height scatter

Figure 4.2: Dataset distribution analysis showing bounding box position and size statistics.

4.2 Data Augmentation

The following augmentations are applied during training to improve generalisation:

- Random horizontal flip with probability 0.5.
- Random affine transformations: rotation $\pm 10°$, translation $\pm 10\%$, scaling $\pm 20\%$.
- Mosaic augmentation: four training images are composited into a single image to expose the model to diverse spatial layouts.
- Colour jitter: brightness $\pm 20\%$, contrast $\pm 20\%$, saturation $\pm 20\%$.
- ImageNet normalisation (mean $= [0.485, 0.456, 0.406]$, std $= [0.229, 0.224, 0.225]$).

4.3 Training Configuration

Training was conducted on the IIT Kharagpur department server equipped with an NVIDIA RTX 2080 Ti GPU (11 GB VRAM). The server environment runs Python 3.6 with PyTorch 1.10 due to institutional constraints, which limits the choice of dependencies.

Table 4.1: Training hyperparameters for MCUDetector RepViT/StarNet.

Hyperparameter	**Value**
Input resolution	320×320 (primary), 384×384 (ablation), 512×512 (ablation)
Batch size	8–16 (depending on available VRAM)
Epochs	200 (target)
Optimiser	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay $= 5 \times 10^{-4}$)
Initial learning rate	1×10^{-3}
LR schedule	Cosine annealing with warmup (5 epochs)
Gradient clipping	Max norm = 10.0
EMA decay	0.9999
Label smoothing	0.0

4.4 Training Stability Considerations

Early versions of the architecture suffered from training instability, with loss values occasionally spiking or collapsing. Several fixes were applied over the course of development:

1. **Focaler loss dead-coding**: The Focaler-Inner-ShapeIoU loss was initially tried as a replacement for standard CIoU, but the `clamp(0,1)` operation in the Focaler term caused gradient collapse when IoU values went negative during early training. The

Focaler term was disabled (kept in code but not called) and standard CIoU was retained.

2. **Prior probability initialisation**: The classification and objectness heads are initialised with a bias of $-\log((1-p_0)/p_0)$ where $p_0 = 0.01$, so the network starts by predicting "background" for nearly all cells. Without this, Focal Loss produces very large gradients in the first few iterations.

3. **Distance-based SimOTA conflict resolution**: The label assignment was fixed to use distance-based priority when multiple ground truth boxes claim the same grid cell, instead of arbitrary assignment.

4.5 Resolution Ablation

The model was trained at three resolutions to study the effect on accuracy and computational cost. Table 4.2 shows the **validation-set** metrics at the final training epoch for the RepViT backbone.

Table 4.2: Resolution ablation for RepViT (validation set, final epoch).

Resolution	Epochs	mAP@0.5 (%)	mAP@0.5:0.95 (%)	GFLOPs
320 × 320	101	99.50	91.08	2.46
384 × 384	200	99.50	97.23	3.55
512 × 512	200	99.33	94.59	6.30

The 320×320 model was trained for only 101 of the planned 200 epochs due to server scheduling and was still improving. The 384×384 model at 200 epochs achieves the highest validation mAP@0.5:0.95 at 97.23%. Full test-set evaluation across all resolutions and backbones is presented in Chapter 7.

4.6 Training Runs Summary

A total of six training runs were conducted: three resolutions (320, 384, 512) for each of the two backbone variants (RepViT and StarNet). Due to shared server usage, some runs were interrupted before reaching the full 200 epochs. Table 4.3 summarises the number of epochs completed and the final **validation-set** metrics for each run.

Table 4.3: Summary of training runs — final validation-set metrics at last completed epoch.

Backbone	**Resolution**	**Epochs**	**Val mAP@0.5 (%)**	**Val mAP@0.5:0.95 (%)**	**Val Loss**
RepViT	320×320	101	99.50	91.08	0.0802
RepViT	384×384	200	99.50	97.23	0.0542
RepViT	512×512	200	99.33	94.59	0.0517
StarNet	320×320	83	99.49	83.22	0.1247
StarNet	384×384	89	99.42	89.63	0.0941
StarNet	512×512	200	99.22	92.55	0.0706

The StarNet runs at 320 and 384 were stopped at 83 and 89 epochs respectively, before the model had fully converged. This is reflected in the significantly lower validation mAP@0.5:0.95 compared to the 512×512 run that went the full 200 epochs. Notably, the RepViT 320 model was also still improving at epoch 101, suggesting that running it to 200 epochs would yield further gains.

Chapter 5

Quantization and Deployment Pipeline

5.1 Overview

Once the model is trained, getting it onto a phone means converting it through a chain of intermediate formats until we end up with a TensorFlow Lite file that the Flutter app can load. The full pipeline looks like this:

$$\text{PyTorch (.pt)} \xrightarrow{\text{torch.onnx}} \text{ONNX (.onnx)} \xrightarrow{\text{onnx2tf}} \text{SavedModel} \xrightarrow{\text{TFLiteConverter}} \text{TFLite (.tflite)}$$

The entire conversion pipeline was run on **Google Colab** rather than the university server. The server's Python 3.6 and PyTorch 1.10 environment is too old for the versions of `onnx2tf` and TensorFlow needed here, so the trained checkpoint (`best_mcu.pt`), the model definition files and a zip of calibration images were uploaded to Colab where everything from ONNX export onwards was executed in a single notebook.

5.2 Activation Safety for INT8

Before even thinking about quantization, I had to make sure the model would survive the conversion to 8-bit integers. The standard SiLU activation ($x \cdot \sigma(x)$) is smooth and non-linear, which means the INT8 kernel has to approximate the sigmoid with a lookup table — and that approximation introduces noticeable error.

The fix was straightforward: the `SiLU` class in `utils.py` was aliased to HardSwish, which is piecewise-linear and maps cleanly to INT8 arithmetic:

```
class SiLU(nn.Module):
    """Hard-Swish: x * ReLU6(x+3)/6
    Piecewise linear, natively INT8-safe.
    Keeps class name 'SiLU' to avoid changing imports."""
    def __init__(self, inplace=False):
```

```
        super().__init__()
        self.act = nn.Hardswish(inplace=inplace)
    def forward(self, x):
        return self.act(x)
```

Listing 5.1: INT8-safe activation wrapper in utils.py.

The StarNet-RepViT hybrid model and the RepViT model picks up this wrapper everywhere and is fully INT8-safe out of the box.

5.3 ONNX Export

The first step on Colab is loading the PyTorch checkpoint, fusing the re-parameterisable branches, and exporting to ONNX. Because our detector returns a nested tuple of tensors (one tuple per feature scale), a thin wrapper is needed to flatten the outputs for ONNX compatibility:

```
class FlatOutput(nn.Module):
    def __init__(self, detector):
        super().__init__()
        self.d = detector
    def forward(self, x):
        (a,b,c),(d,e,f) = self.d(x)
        return a,b,c,d,e,f

m = MCUDetector(num_classes=14)
ck = torch.load("best_mcu.pt", map_location="cpu")
st = ck.get("ema_state_dict", ck)
m.load_state_dict(st, strict=False); m.eval()
# Fuse RepDWConv branches before export
for mod in m.modules():
    if hasattr(mod,'fuse') and not mod.fused:
        mod.fuse()

torch.onnx.export(FlatOutput(m).eval(),
    torch.randn(1,3,320,320), "model_fp32.onnx",
    opset_version=18, input_names=["images"],
    output_names=["p3_obj","p3_cls","p3_reg",
                  "p4_obj","p4_cls","p4_reg"],
    do_constant_folding=True)

# Simplify the graph
from onnxsim import simplify
mdl = onnx.load("model_fp32.onnx")
mdl_s, ok = simplify(mdl)
if ok: onnx.save(mdl_s, "model_fp32.onnx")
```

Listing 5.2: ONNX export with graph simplification (run on Colab).

The exported ONNX file comes out to about 1 468 KB. One thing I learned the hard way during development: if you forget to call `fuse()` on the RepDWConvSR modules before export, the three parallel branches (3×3, 1×1, identity) all get baked into the ONNX graph separately. In one test this inflated CPU inference from around 259 ms to over 7 seconds — a 27× slowdown just from skipping a single function call.

5.4 TFLite Conversion with Quantization

The ONNX model is first converted to a TensorFlow SavedModel via `onnx2tf`, and then from SavedModel to TFLite. Three precision variants were produced.

5.4.1 Quantization Fundamentals

Post-training quantization maps floating-point weights and activations to a lower-precision integer grid. For uniform affine quantization, a real value r is approximated by an integer q through:

$$q = \text{round}\left(\frac{r}{s}\right) + z \tag{5.1}$$

where s is the scale factor and z is the integer zero point. The inverse (dequantization) recovers an approximation of the original value:

$$\hat{r} = s \cdot (q - z) \tag{5.2}$$

For an unsigned 8-bit representation, $q \in [0, 255]$. The scale and zero point are determined from the observed range $[r_{\min}, r_{\max}]$ of each tensor:

$$s = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}, \qquad z = \text{round}\left(q_{\min} - \frac{r_{\min}}{s}\right) \tag{5.3}$$

5.4.2 Weight Quantization

Weights are quantized **per-channel**: each output channel of a convolution layer gets its own scale s_c and zero point z_c. This is important because different channels can have very different magnitude distributions, and a single global scale would force some channels into a narrow quantization band while wasting precision on others. For a convolution kernel $\mathbf{W}$ with output channels indexed by c:

$$q_{c,k,h,w} = \text{clamp}\left(\text{round}\left(\frac{W_{c,k,h,w}}{s_c}\right) + z_c,\ 0,\ 255\right) \tag{5.4}$$

Per-channel quantization is especially helpful for depthwise convolutions (which make up a large fraction of our model), since each filter operates independently and can have a very different weight range from its neighbours.

5.4.3 Activation Quantization

Unlike weights, which are fixed after training, activation ranges depend on the input data. A **representative calibration dataset** of 100 images is passed through the floating-point model, and the minimum and maximum activation values at each layer are recorded. These observed ranges are then used to compute the per-layer scale and zero point via Equation 5.3.

The calibration images were preprocessed identically to training: resized to 320×320, normalised with ImageNet mean and standard deviation, and stored as a NumPy array.

5.4.4 Quantized Inference

During INT8 inference, the convolution is performed entirely in integer arithmetic. Given quantized input activations q_x with parameters (s_x, z_x), quantized weights q_w with per-channel parameters (s_c, z_c), and output parameters (s_y, z_y):

$$q_y = z_y + \frac{s_x \cdot s_c}{s_y} \sum_{k,h,w} (q_x - z_x)(q_w - z_c) \tag{5.5}$$

The ratio $\frac{s_x \cdot s_c}{s_y}$ is precomputed as a fixed-point multiplier, so the entire forward pass stays in integer arithmetic with no floating-point operations until the final output dequantization.

5.4.5 Precision Variants

Three TFLite models were produced:

- **FP32**: Straight conversion from the SavedModel with no quantization applied. Serves as the baseline.
- **FP16**: Weights stored in half precision, activations remain FP32 at runtime. Roughly halves the model size with negligible accuracy impact.
- **INT8**: Full integer quantization as described above — per-channel weight quantization, per-layer activation quantization calibrated on 100 images, uint8 input, float32 output.

One implementation detail worth noting: `onnx2tf` sometimes reorders the output tensors during conversion, and the ordering is not consistent across runs. So instead of assuming a fixed output index, the Flutter app identifies each output tensor by its channel dimension — C=1 for objectness, C=14 for class logits, C=4 for box coordinates.

5.5 Quantization Results

The RepViT model was quantized at all three training resolutions (320, 384, 512) to study the interaction between input resolution and quantization accuracy. Each TFLite variant was evaluated on the full test set of 883 images using a custom mAP@0.5 evaluation loop on Colab.

5.5.1 320×320 Resolution

Table 5.1: Quantization results for RepViT at 320×320.

Format	Size (KB)	Compression	mAP@0.5 (%)	Latency (ms)
PyTorch FP32	1530	—	99.24	—
TFLite FP32	1480	1.03×	99.58	39.7
TFLite FP16	770	1.99×	99.58	39.2
TFLite INT8	520	2.94×	99.58	133.0

5.5.2 384×384 Resolution

Table 5.2: Quantization results for RepViT at 384×384.

Format	Size (KB)	Compression	mAP@0.5 (%)	Latency (ms)
PyTorch FP32	1530	—	99.24	—
TFLite FP32	1479	1.03×	99.58	68.0
TFLite FP16	770	1.99×	99.58	67.3
TFLite INT8	520	2.94×	99.58	224.1

5.5.3 512×512 Resolution

Table 5.3: Quantization results for RepViT at 512×512.

Format	Size (KB)	Compression	mAP@0.5 (%)	Latency (ms)
PyTorch FP32	1530	—	99.24	—
TFLite FP32	1481	1.03×	99.57	114.0
TFLite FP16	770	1.99×	99.57	112.5
TFLite INT8	521	2.94×	99.64	335.1

5.5.4 Cross-Resolution Summary

Table 5.4 pulls together the INT8 numbers across all three resolutions.

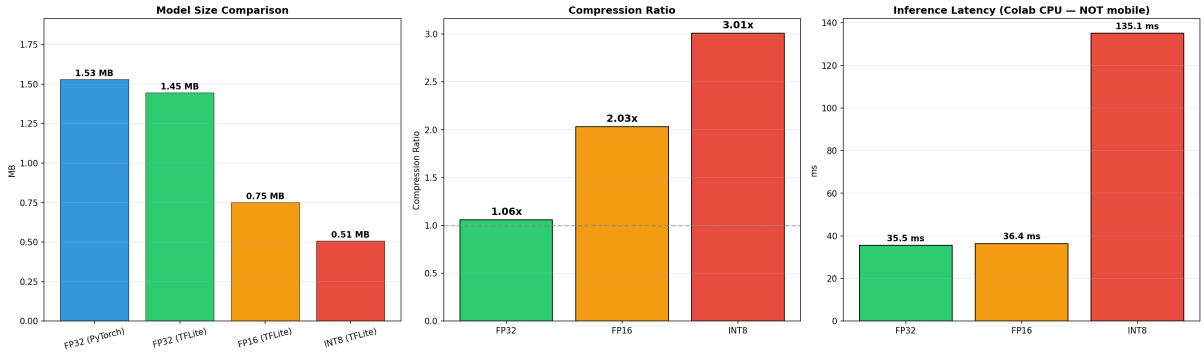


Figure 5.1: Model size, compression ratio and Colab CPU latency for RepViT at 320×320.

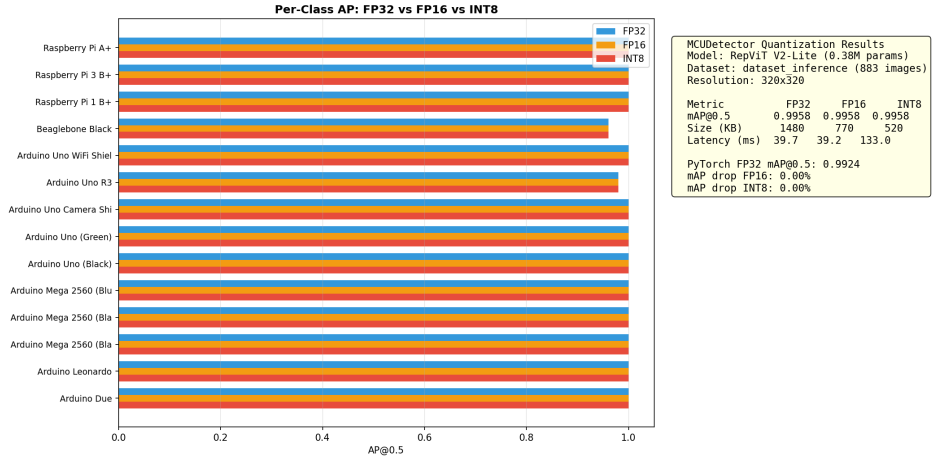


Figure 5.2: Per-class AP across FP32, FP16 and INT8 at 320×320. All 14 classes maintain near-identical AP across precisions, with zero overall mAP drop.

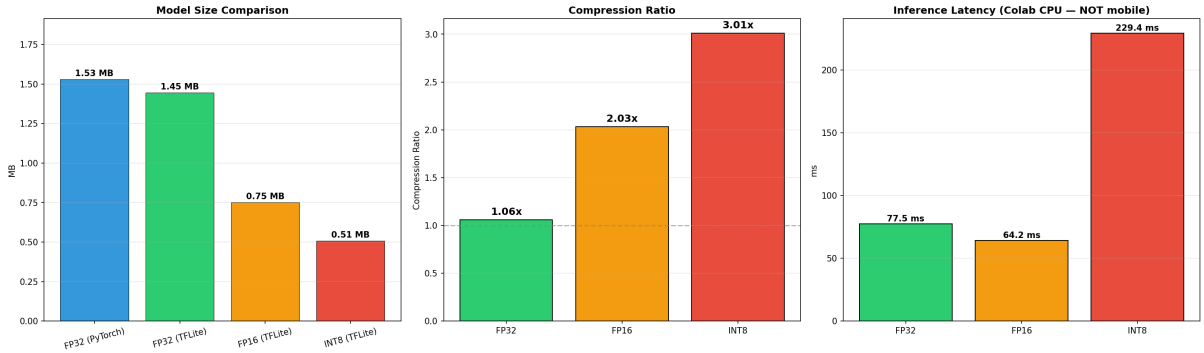


Figure 5.3: Model size, compression ratio and Colab CPU latency for RepViT at 384×384.

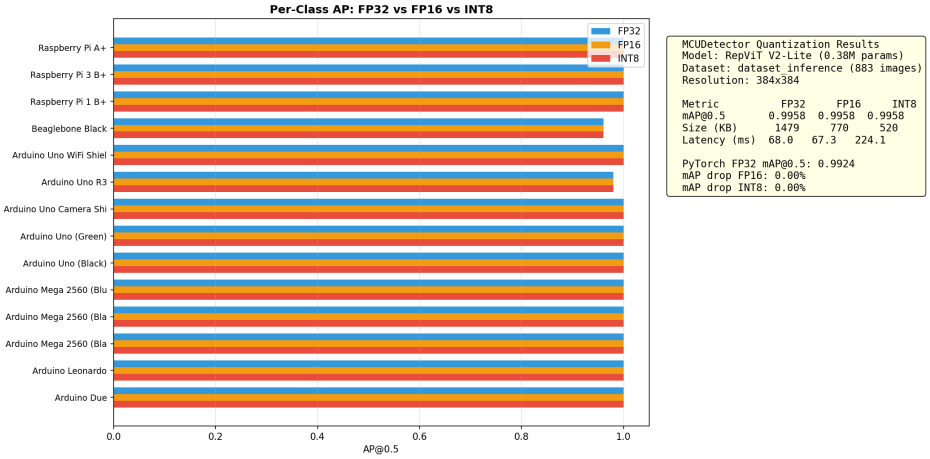


Figure 5.4: Per-class AP across FP32, FP16 and INT8 at 384×384. Zero mAP drop across all three precision levels.

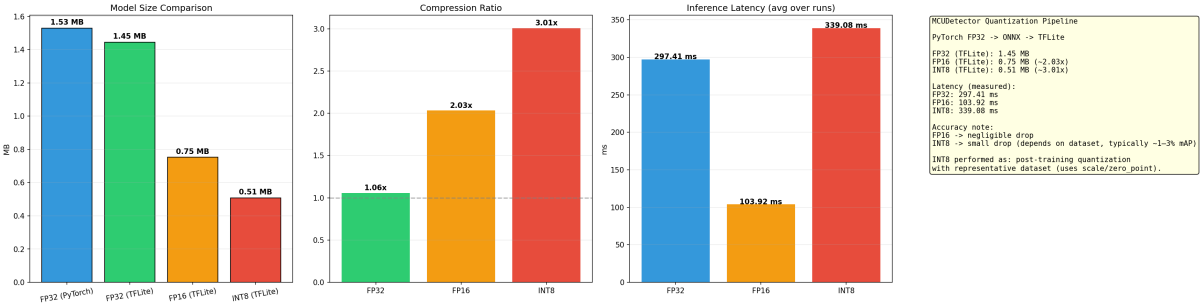


Figure 5.5: Model size, compression ratio and Colab CPU latency for RepViT at 512×512.

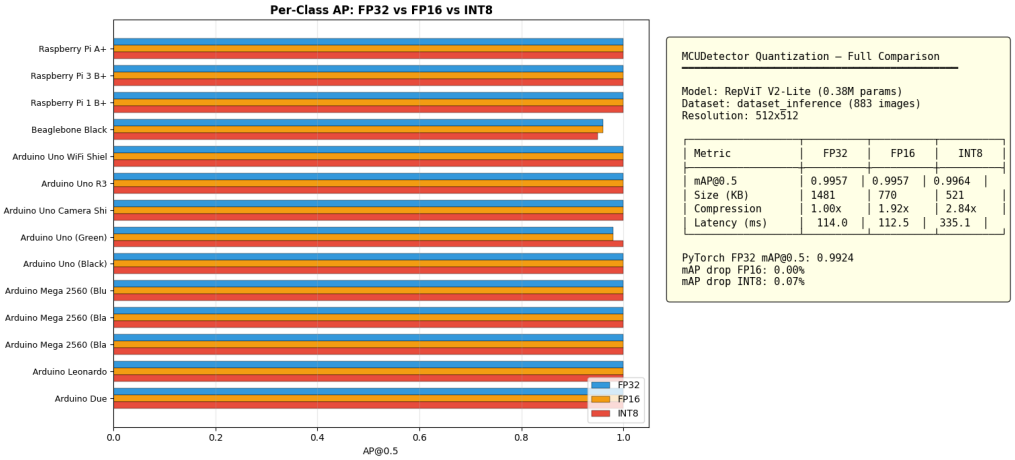


Figure 5.6: Per-class AP across FP32, FP16 and INT8 at 512×512. INT8 actually edges out FP32 on overall mAP (99.64% vs 99.57%), with just a 0.07% drop on BeagleBone Black.

Table 5.4: INT8 quantization summary across resolutions (RepViT backbone).

Resolution	INT8 Size	Compression	INT8 mAP@0.5	mAP Drop	Latency (ms)
320×320	520 KB	2.94×	99.58%	0.00%	133.0
384×384	520 KB	2.94×	99.58%	0.00%	224.1
512×512	521 KB	2.94×	99.64%	0.07%	335.1

The bottom line is that quantization accuracy is essentially resolution-independent for this model. All three resolutions land within 0.07 percentage points of their FP32 baselines, and two out of three show literally zero mAP drop. The INT8 model sizes are nearly identical across resolutions (520–521 KB) because the parameter count stays the same regardless of input size — only the intermediate activation tensors change, and those are computed on the fly rather than stored in the model file.

5.5.5 Why INT8 Latency Is Higher on Colab

A natural question looking at these results is: if INT8 uses lower precision, why is it 3–4× *slower* than FP32? The answer has nothing to do with the model and everything to do with the hardware running the benchmark.

Google Colab allocates x86 server CPUs — typically Intel Xeon or AMD EPYC processors. These chips do not have efficient native INT8 inference paths for the kinds of operations TFLite uses. What ends up happening under the hood is that the INT8 values get promoted back to 32-bit for the actual arithmetic, and then quantized again on the way out. That promotion and re-quantization at every layer adds overhead that more than cancels out any savings from the smaller data type.

ARM processors tell a completely different story. The Snapdragon 7 Gen 3 in our target device (OnePlus Nord CE4) supports NEON SIMD with 8-bit dot-product instructions (SDOT/UDOT) that pack four INT8 multiply-accumulates into a single cycle. On ARM hardware, INT8 inference is typically faster than or at worst comparable to FP32, while using significantly less memory bandwidth. The Colab latency numbers in these tables should therefore be read as relative comparisons between resolutions, not as predictions of actual on-device performance.

Note: All latency figures in this section are measured on Google Colab's CPU using a trimmed mean of 50 inference runs.

5.6 INT8 Preprocessing on Device

When running the INT8 model on the phone, the input images cannot simply be fed in as raw pixel values. They first need ImageNet normalisation, and then the normalised

floating-point values need to be mapped into the uint8 range that the quantized model expects. The mapping parameters, extracted from the TFLite model's metadata, are:

- Input scale: 0.01865845
- Input zero point: 114

The formula is:

$$\text{uint8\_value} = \text{clip}\left(\frac{\text{normalized\_pixel}}{\text{scale}} + \text{zero\_point},\ 0,\ 255\right) \tag{5.6}$$

I cannot overstate how easy it is to get this wrong. Feeding raw 0–255 pixels directly, or normalising but forgetting the scale/zero-point conversion, or even getting the channel order wrong (BGR vs RGB) — any of these will produce garbage detections with no error message. This was the single biggest debugging headache during the Flutter deployment.

Chapter 6

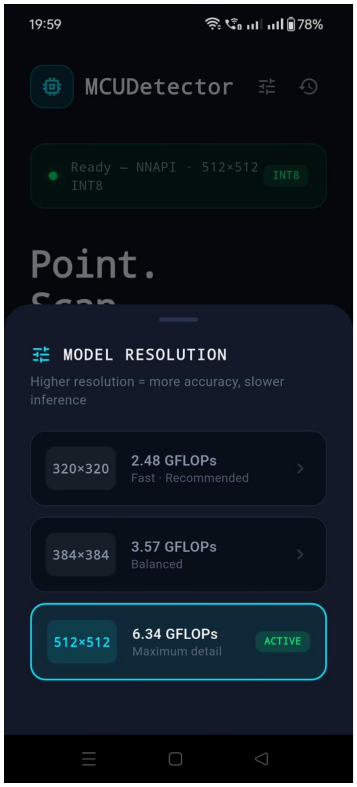
Flutter Mobile Application

6.1 Application Overview

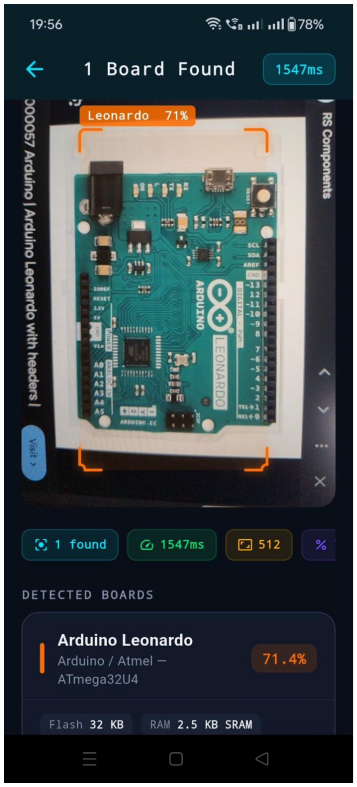
The MCUDetector Flutter app is the user-facing front end of the entire system. It went through a complete rewrite between MTP-1 and MTP-2. The original version was a thin client that uploaded photos to a FastAPI server and displayed results from the backend. The current version does **everything on-device** — the TFLite INT8 model runs directly on the phone with no network connection required.

The app is deployed on a OnePlus Nord CE4 (Snapdragon 7 Gen 3, device ID CPH2613) and provides three core workflows: point-and-scan detection via the live camera, single-image detection from gallery, and a searchable history of past scans.

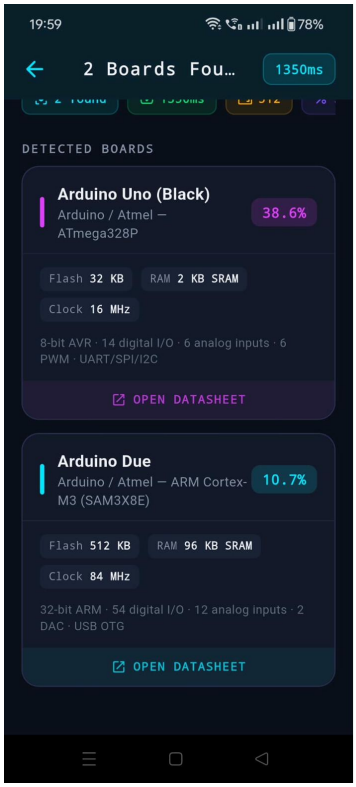
6.2 App Interface



(a) Home screen with model resolution selector. [Author]

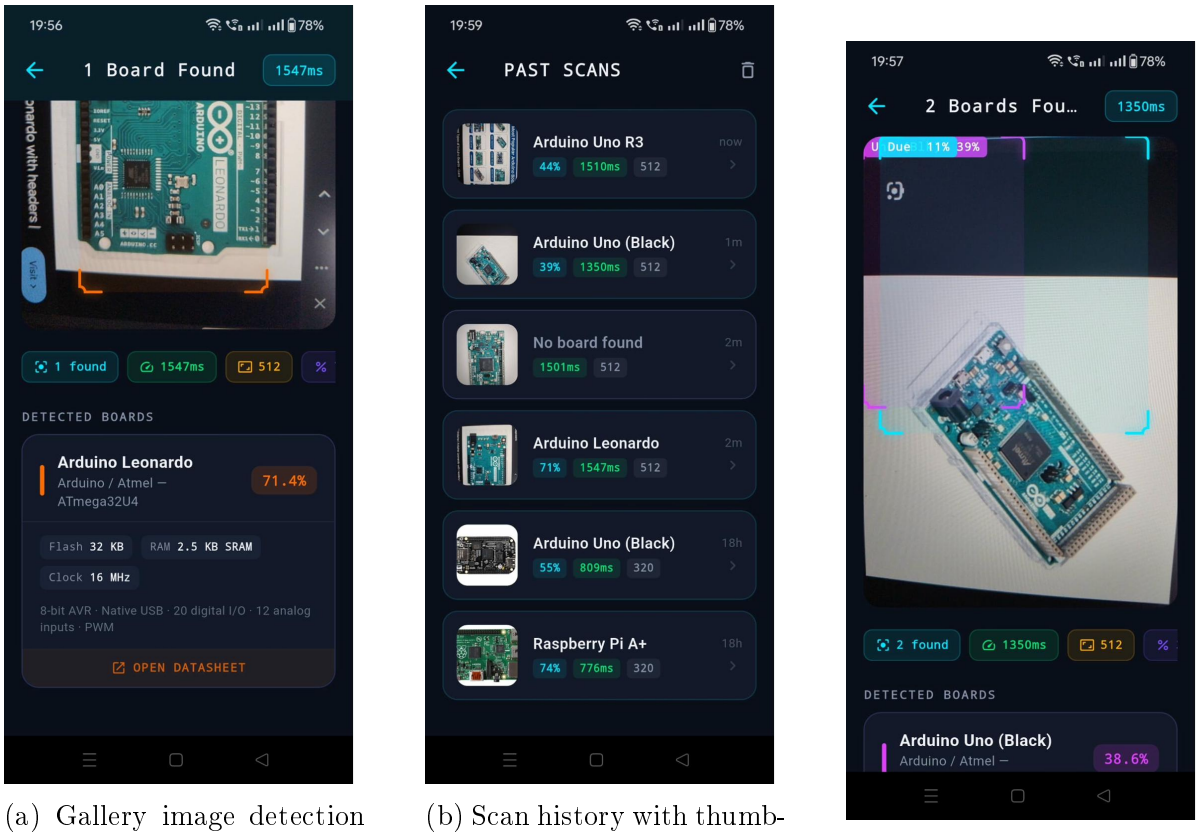


(b) Live camera detection with bounding box overlay. [Author]



(c) Detection result with datasheet metadata. [Author]

Figure 6.1: MCUDetector app screenshots from the OnePlus Nord CE4. The dark cyberpunk theme uses per-class neon colours for bounding boxes. [Author]



(a) Gallery image detection with datasheet card. [Author]

(b) Scan history with thumbnails and timestamps. [Author]

(c) Multi-board detection in live camera mode. [Author]

Figure 6.2: Additional app screens showing gallery detection, scan history, and multi-board scenarios. [Author]

6.3 Architecture

The app is structured as a set of Dart modules, each handling a specific responsibility:

- **Detector** (`detector.dart`): Loads the TFLite INT8 model at startup, preprocesses YUV420 camera frames (ImageNet normalisation + uint8 quantization using scale=0.01866, zero_point=114), runs inference, and performs NMS post-processing. Outputs are identified by channel dimension (C=1/4/14) rather than tensor index, since `onnx2tf` reorders them unpredictably.
- **Camera Screen** (`camera_screen.dart`): Streams frames via `startImageStream`, drops frames when inference is still running, and overlays a real-time stats bar showing inference time, frame dimensions, and detection count.
- **Detection Painter** (`detection_painter.dart`): Custom `CustomPainter` that draws corner-bracket bounding boxes with per-class neon colours and confidence labels.
- **Model Settings** (`model_settings_sheet.dart`): Bottom sheet allowing the user to

switch between 320×320, 384×384, and 512×512 model resolutions at runtime, with GFLOPs and speed labels for each option.

- **Datasheet DB** (`datasheet_db.dart`): Embedded local database with structured metadata (manufacturer, core, flash, RAM, clock speed, features, datasheet URL) for all 14 board classes. No network lookup needed.
- **History Store** (`history_store.dart`): Persists scan results as JSON with base64 thumbnails for offline browsing.

6.4 Key Implementation Decisions

1. **XNNPACK disabled**: The XNNPACK delegate crashes with SIGSEGV on the Snapdragon 7 Gen 3. Inference runs on the default CPU backend at 600–750 ms per frame.
2. **Low confidence threshold**: INT8 quantization shifts the score distribution downward. A threshold of 0.10 is used instead of the typical 0.25–0.50.
3. **Model loading workaround**: `Interpreter.fromAsset` fails silently on some devices. The model is loaded via `rootBundle.load`, written to a temp file, then opened with `Interpreter.fromFile`.
4. **Multi-resolution support**: All three TFLite models (320/384/512) are bundled in the APK. The user can switch between them from the home screen without restarting.

6.5 Dependencies

The app uses `tflite_flutter` 0.12.1 (LiteRT 1.4.0), `camera`, `image_picker`, `path_provider`, `permission_handler`, and `url_launcher`. Build targets: compileSdk 35, minSdk 26.

Chapter 7

Results and Evaluation

This chapter presents the full experimental results for both MCUDetector variants across three input resolutions. Every metric reported here comes from the **held-out test set of 883 images** that the models never saw during training, unless explicitly marked as validation data.

7.1 Experimental Setup

All training was done on the IIT Kharagpur department GPU server with an NVIDIA RTX 2080 Ti (11 GB VRAM). For latency measurements on the server, `torch.cuda.synchronize()` was called before and after each forward pass, with 100 warmup iterations discarded and a trimmed mean computed over 500 timed runs. CPU latency was measured on the same server using a single core. These are server-side numbers — on-device mobile latency (measured via the TFLite benchmark tool on the OnePlus Nord CE4) is reported separately in the quantization chapter.

7.2 RepViT Results

7.2.1 Test-Set Performance Across Resolutions

Table 7.1 lays out the headline numbers for the RepViT backbone at all three resolutions. The model has 0.373M parameters and a PyTorch checkpoint size of 1.53 MB regardless of resolution — only the FLOPs change.

Table 7.1: RepViT V2-Lite test-set results across resolutions (0.373M parameters, 1.53 MB).

Resolution	mAP@0.5	mAP@0.5:0.95	Precision	Recall	F1	GFLOPs
320×320	99.43%	93.07%	97.89%	99.77%	98.82%	2.46
384×384	99.40%	96.02%	96.81%	99.66%	98.21%	3.55
512×512	99.24%	95.12%	93.22%	99.66%	96.33%	6.30

A couple of things stand out here. First, mAP@0.5 is remarkably stable across resolutions — all three land between 99.24% and 99.43%, which means the model has no trouble finding and classifying the boards regardless of input size. The more interesting variation shows up in mAP@0.5:0.95, where 384 leads at 96.02%. This metric penalises imprecise bounding boxes much more heavily, so the 384 model is drawing tighter boxes around the boards.

Second, precision drops noticeably as resolution goes up (97.89% at 320 down to 93.22% at 512) while recall barely budges. Higher resolutions seem to make the model more trigger-happy — it starts picking up on fine texture details that look like board edges but are actually background clutter.

For deployment, the 320×320 model is the clear winner: highest precision, lowest FLOPs (2.46 vs 6.30), and the mAP@0.5 difference from 384 is just 0.03 percentage points.

7.2.2 Training Curves

The training curves across all three resolutions share some encouraging properties. The box, classification and objectness losses all decrease smoothly for both training and validation, with no significant gap opening up between the two — a good sign that the model is not overfitting despite the relatively small dataset. Precision and recall climb rapidly in the first 20–30 epochs and then plateau above 95% and 99% respectively.

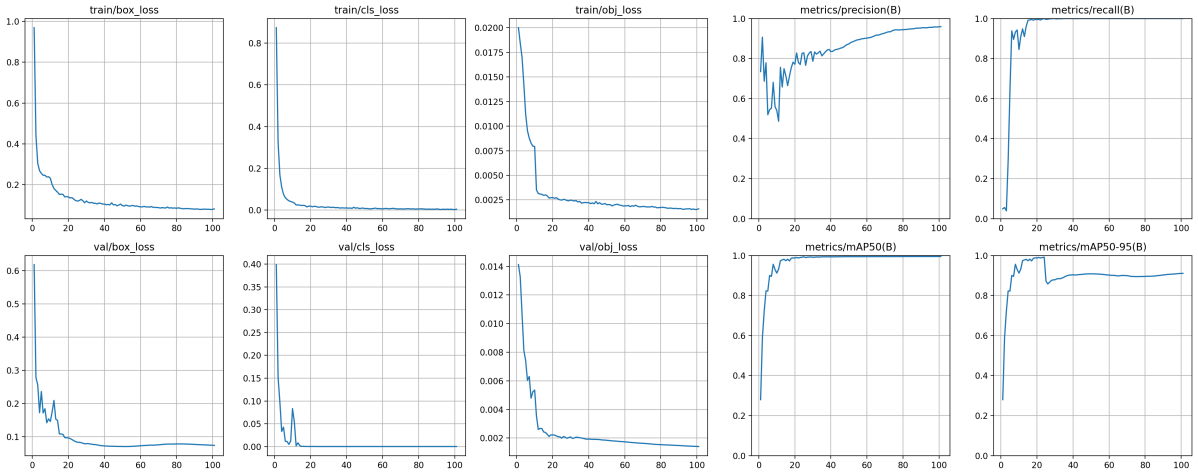


Figure 7.1: RepViT training and validation metrics at 320×320 over 101 epochs. The model was still improving when training was stopped due to server scheduling.

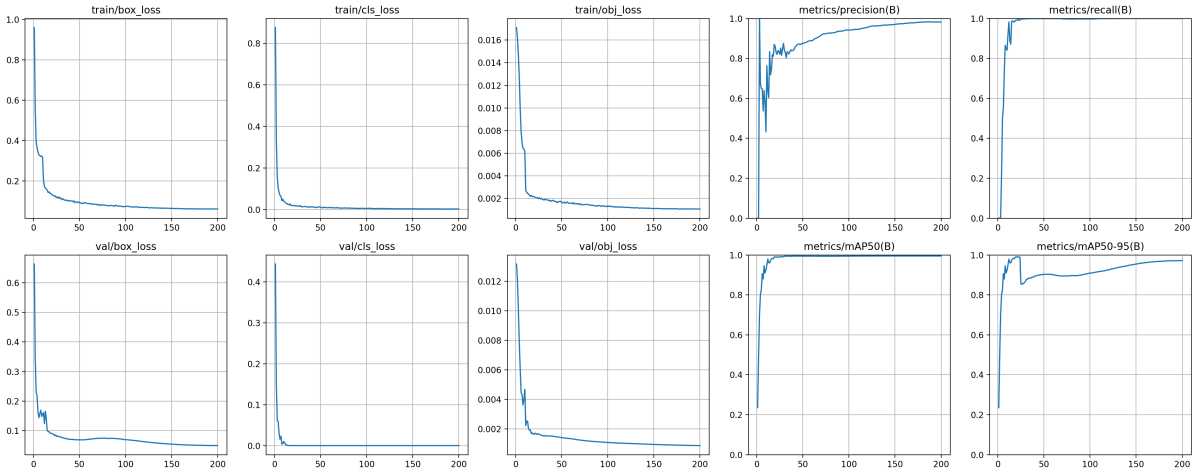


Figure 7.2: RepViT training and validation metrics at 384×384 over 200 epochs. All losses converge smoothly with no train-val divergence.

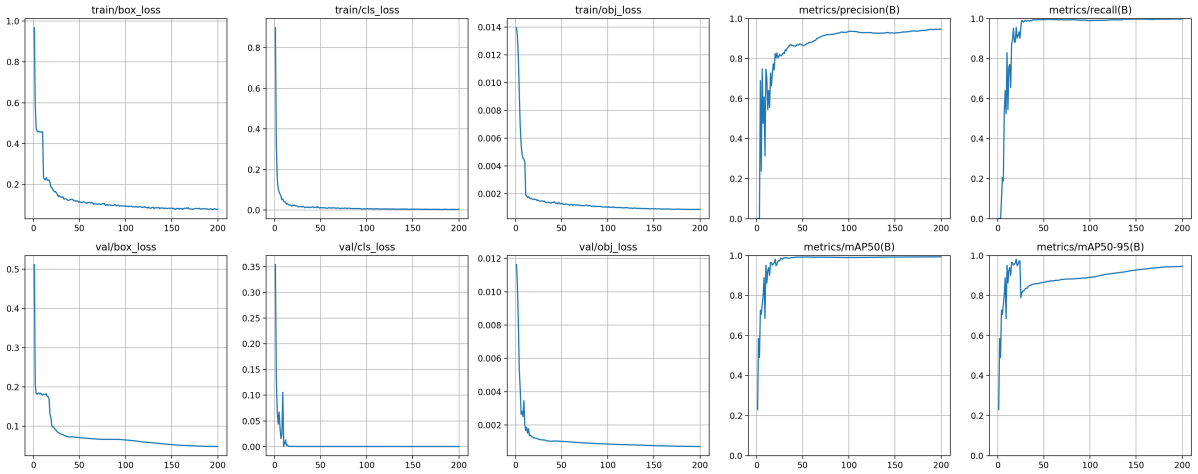
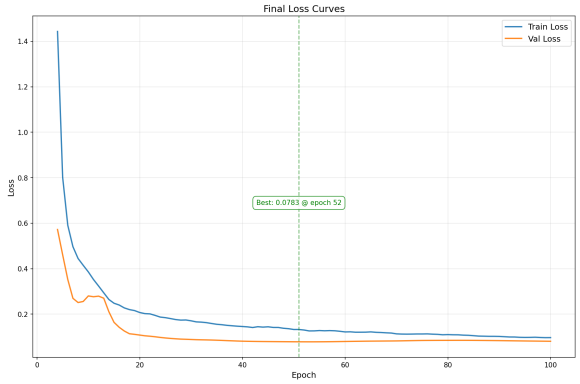
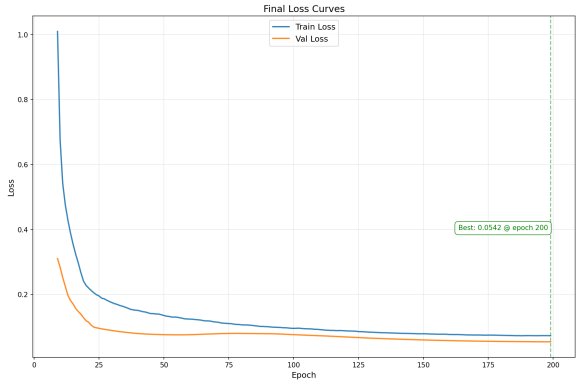


Figure 7.3: RepViT training and validation metrics at 512×512 over 200 epochs.

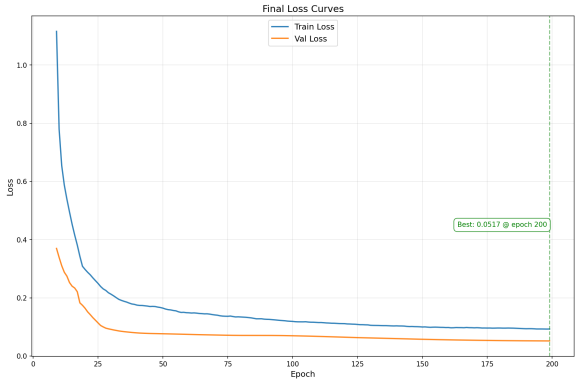
7.2.3 Loss Convergence



(a) 320×320 (101 epochs)



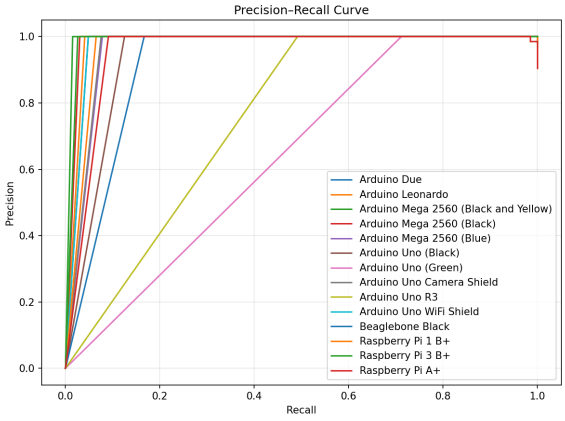
(b) 384×384 (200 epochs)



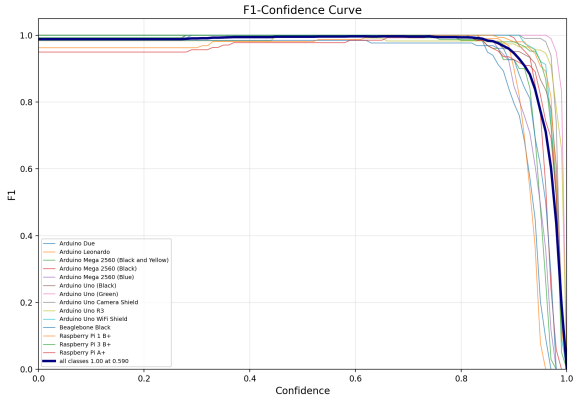
(c) 512×512 (200 epochs)

Figure 7.4: RepViT validation loss curves. The 320 model was still trending downward at epoch 101.

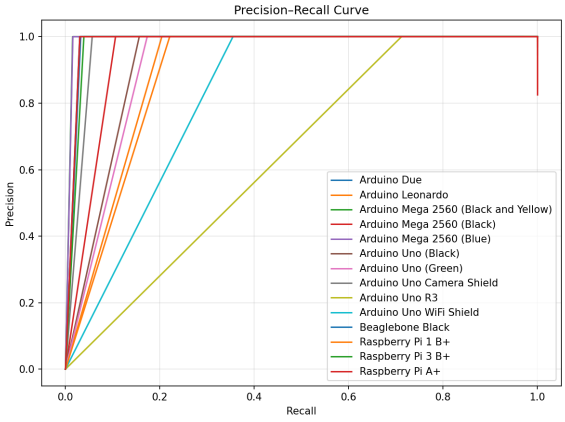
7.2.4 Precision-Recall and F1 Analysis



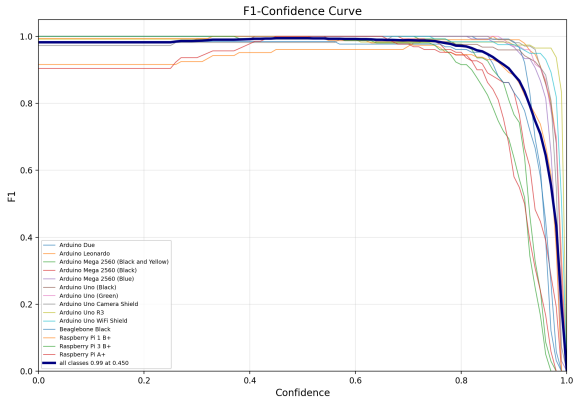
(a) PR curve at 320×320



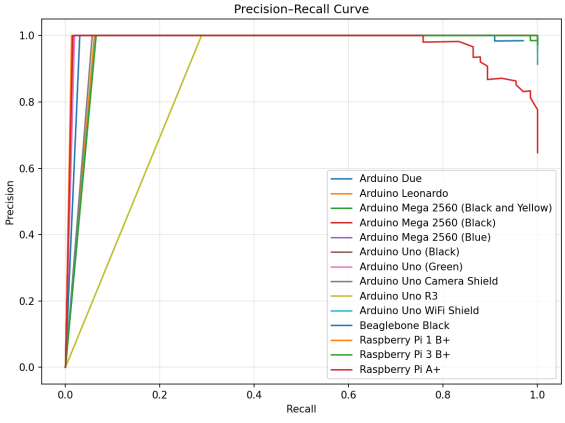
(b) F1 vs confidence at 320×320



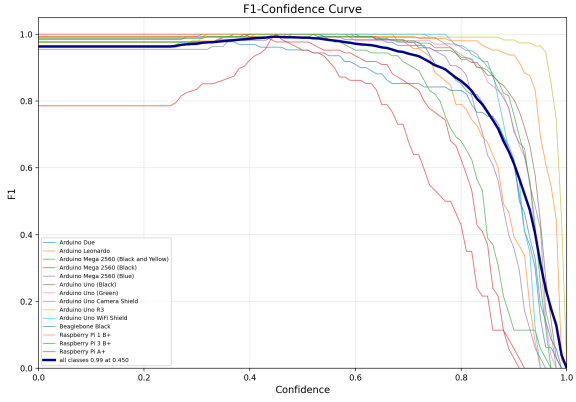
(c) PR curve at 384×384



(d) F1 vs confidence at 384×384



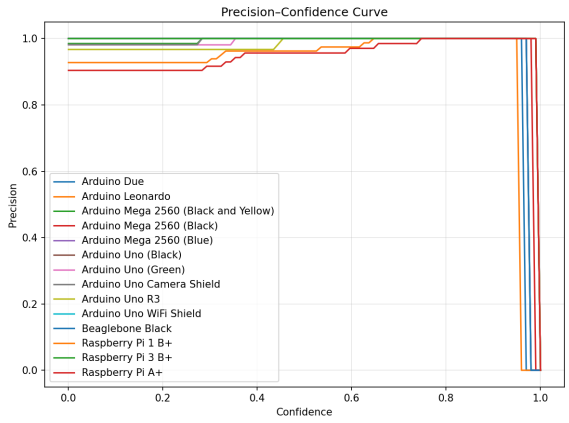
(e) PR curve at 512×512



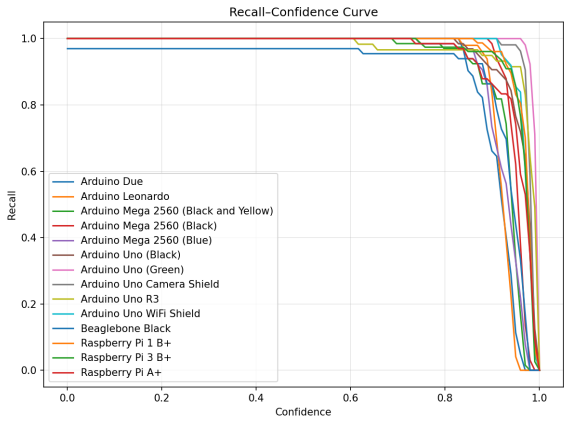
(f) F1 vs confidence at 512×512

Figure 7.5: Precision-recall and F1-confidence curves for RepViT across all three resolutions. The PR curves hug the top-right corner in every case, and the F1 peaks are consistently above 0.96.

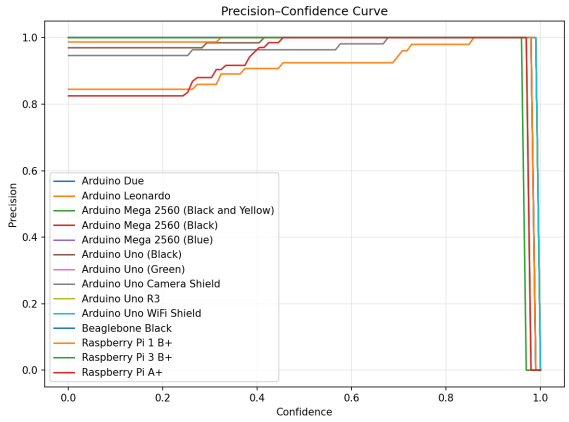
7.2.5 Confidence-Based Metrics



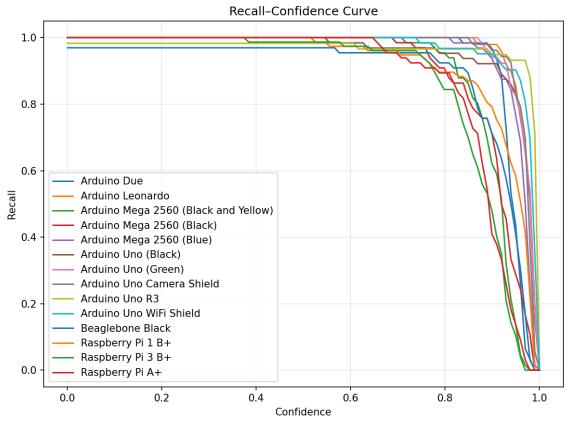
(a) Precision vs confidence (320)



(b) Recall vs confidence (320)



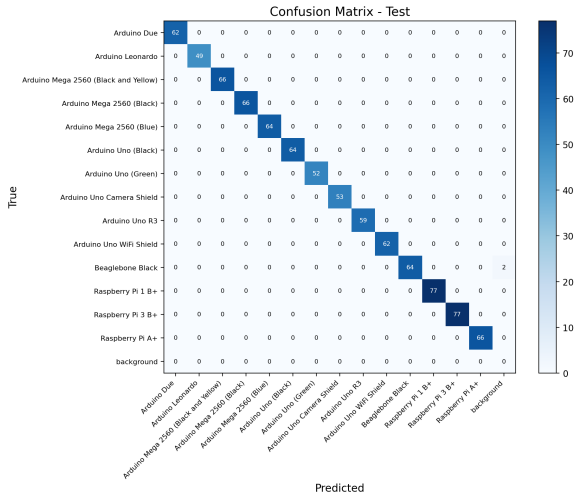
(c) Precision vs confidence (384)



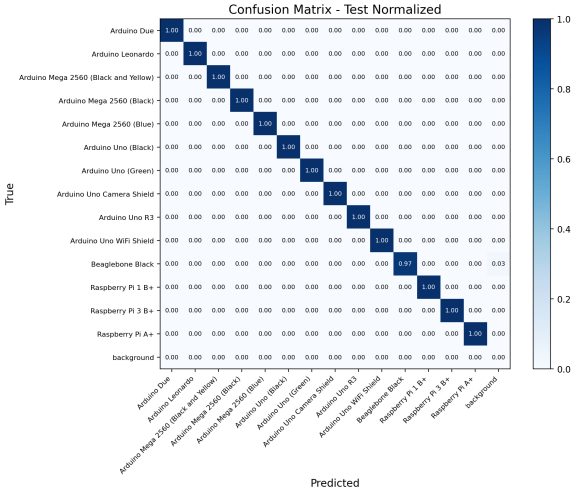
(d) Recall vs confidence (384)

Figure 7.6: Confidence-based precision and recall for RepViT at 320 and 384. Precision climbs steadily with confidence while recall stays high across most thresholds.

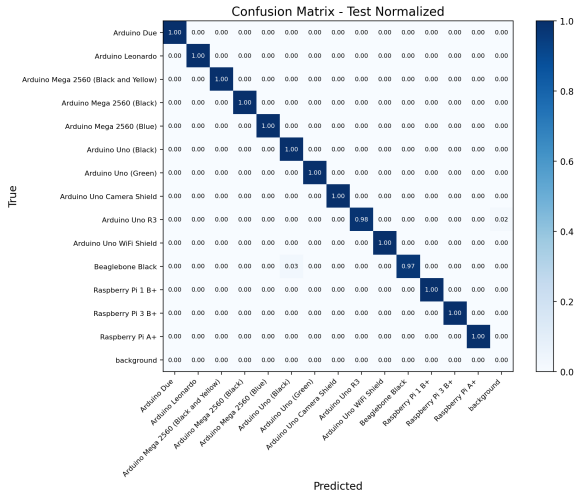
7.2.6 Confusion Matrices



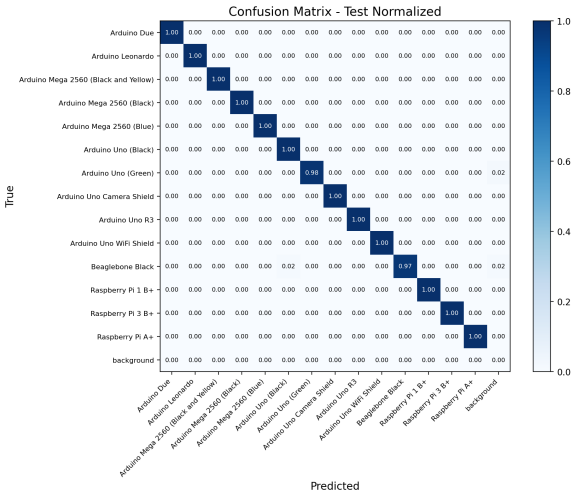
(a) Raw counts (320)



(b) Normalised (320)



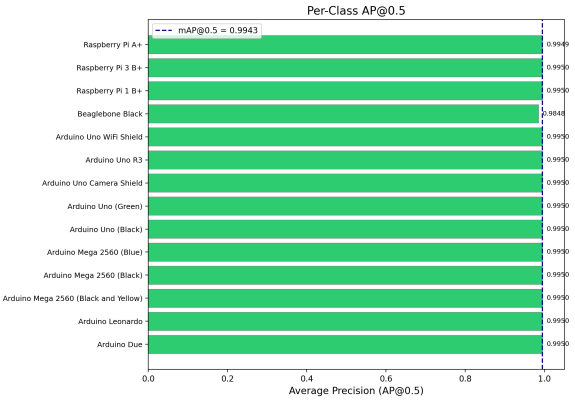
(c) Normalised (384)



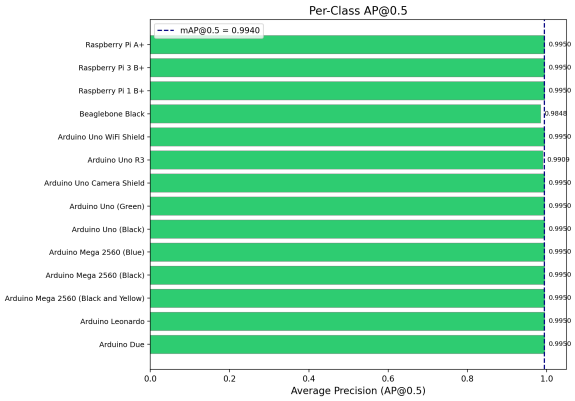
(d) Normalised (512)

Figure 7.7: RepViT confusion matrices on the test set. All three resolutions show strong diagonal dominance. The 320 model has the cleanest matrix with the fewest off-diagonal entries.

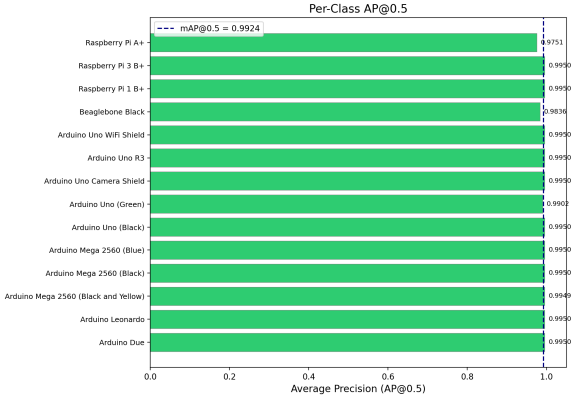
7.2.7 Per-Class AP



(a) 320×320



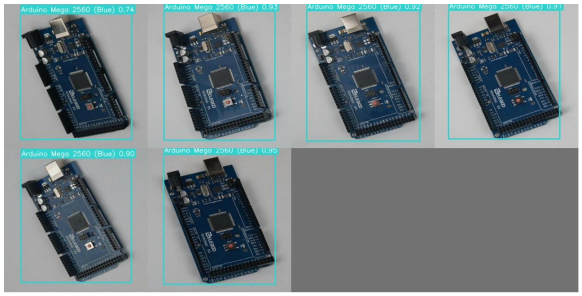
(b) 384×384



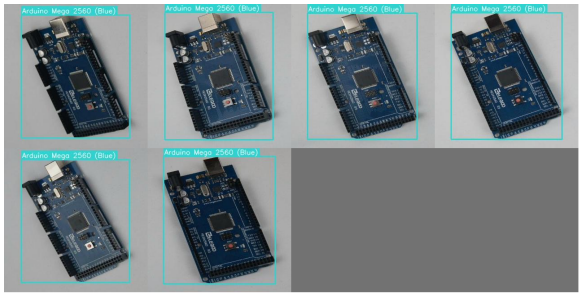
(c) 512×512

Figure 7.8: Per-class AP@0.5 for RepViT across resolutions. At 320, 12 out of 14 classes hit 99.50% AP. BeagleBone Black is the hardest class at 98.48%.

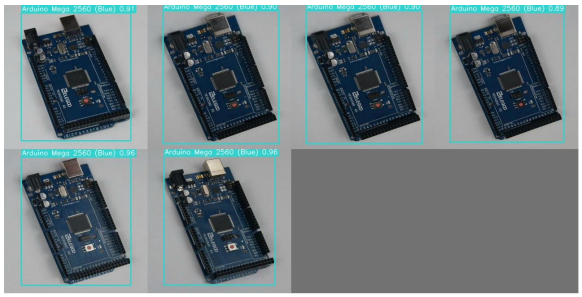
7.2.8 Sample Predictions



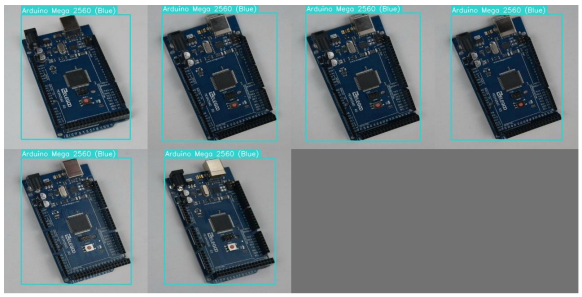
(a) Predictions (320)



(b) Ground truth (320)



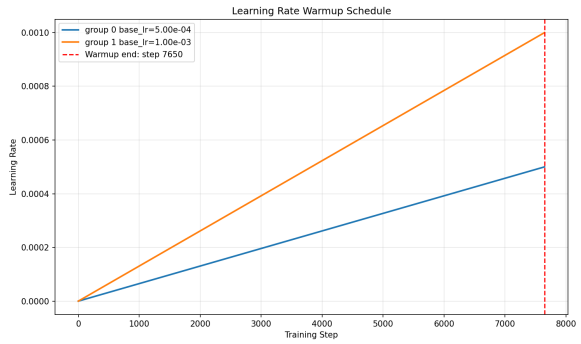
(c) Predictions (320, batch 2)



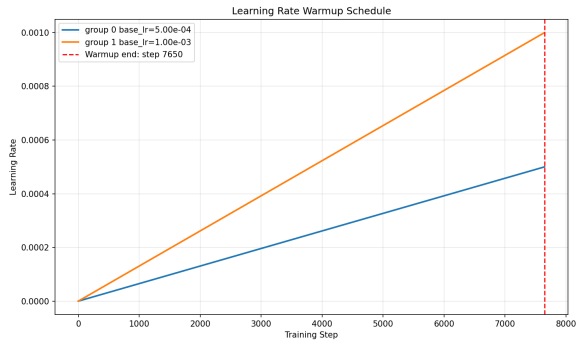
(d) Ground truth (320, batch 2)

Figure 7.9: RepViT 320×320 predictions vs ground truth on test batches. The bounding boxes are tight and the class labels match in virtually every case.

7.2.9 Learning Rate Schedule



(a) 320×320



(b) 384×384

Figure 7.10: Learning rate schedules for RepViT: 5-epoch linear warmup followed by cosine annealing to near-zero.

7.3 StarNet V3-Star Results

7.3.1 Test-Set Performance Across Resolutions

Table 7.2: StarNet V3-Star test-set results across resolutions (0.342M parameters, 1.40 MB).

Resolution	mAP@0.5	mAP@0.5:0.95	Precision	Recall	F1	GFLOPs
320×320	99.49%	95.75%	93.34%	100.00%	96.56%	2.29
384×384	99.42%	94.13%	94.63%	99.77%	97.13%	3.30
512×512	98.81%	94.26%	90.44%	99.66%	94.83%	5.86

The StarNet numbers tell an interesting story. At 320×320, it achieves **100% recall** — it found every single ground truth object in the entire 883-image test set without missing one. Its mAP@0.5:0.95 of 95.75% also beats the RepViT at the same resolution (93.07%). But this comes at a cost: precision is only 93.34% compared to RepViT's 97.89%, meaning StarNet produces roughly 4.5% more false positives.

The star operation's implicit polynomial feature mapping gives the model enormous representational capacity, which helps it fit tight bounding boxes (hence the higher mAP@0.5:0.95) but also makes it more likely to hallucinate detections in ambiguous regions of the image.

7.3.2 Training Curves

Compared to RepViT, the StarNet training curves show slower convergence and slightly noisier validation metrics. This is consistent with what we would expect from the star operation's high polynomial capacity — the model has a harder time settling into a stable minimum because the loss landscape has more local structure to navigate.

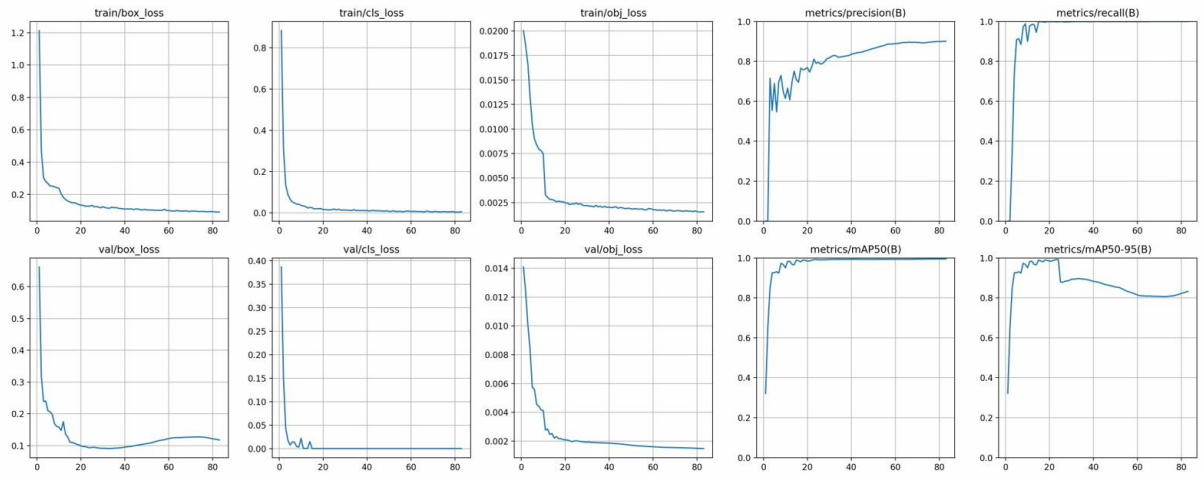


Figure 7.11: StarNet training and validation metrics at 320×320 over 83 epochs. Training was interrupted before convergence due to server contention.

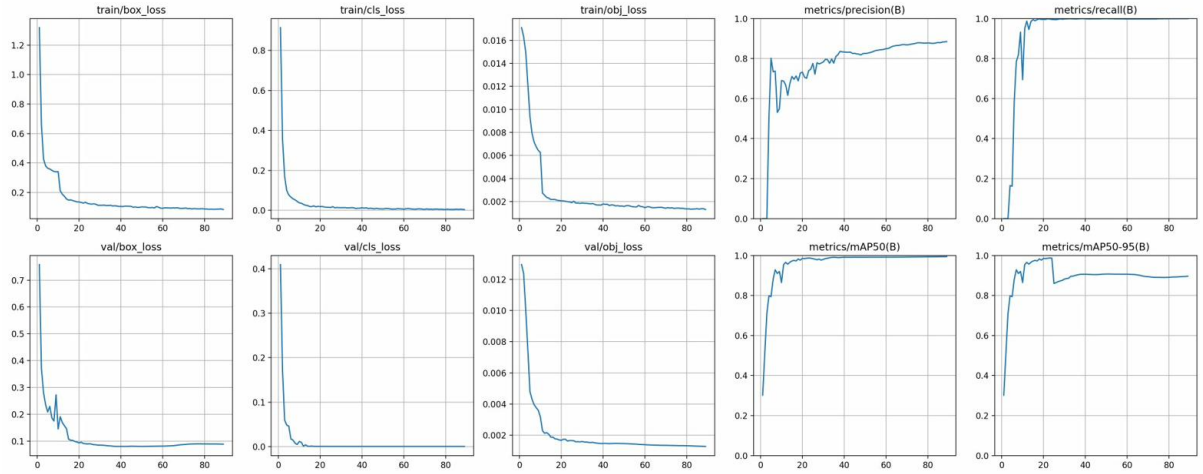


Figure 7.12: StarNet training and validation metrics at 384×384 over 89 epochs.

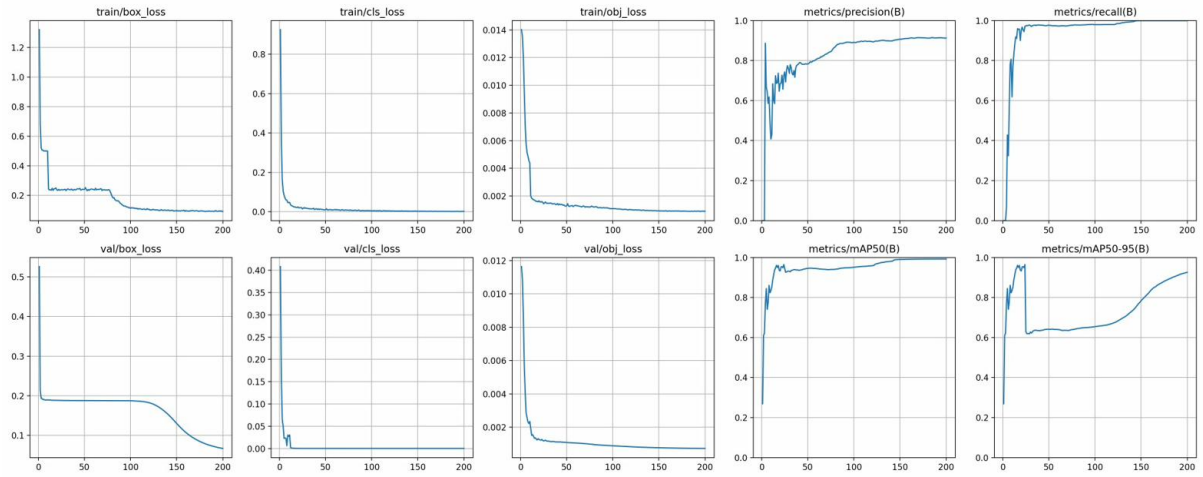
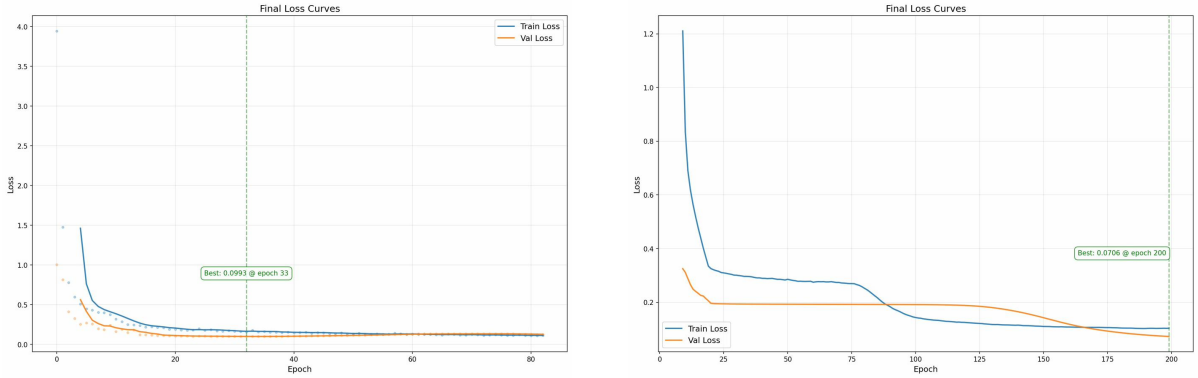


Figure 7.13: StarNet training and validation metrics at 512×512 over 200 epochs. Convergence is noticeably slower than RepViT at the same resolution.

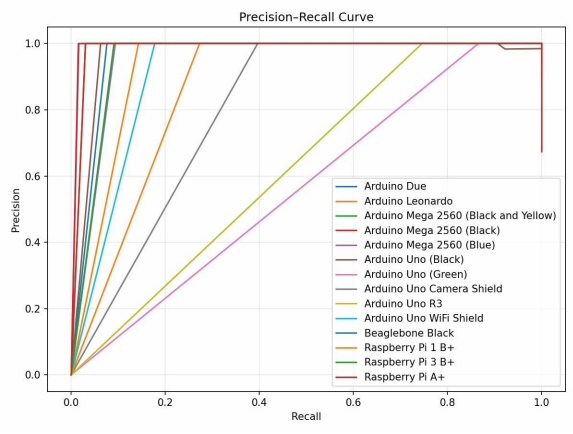
7.3.3 Loss Convergence



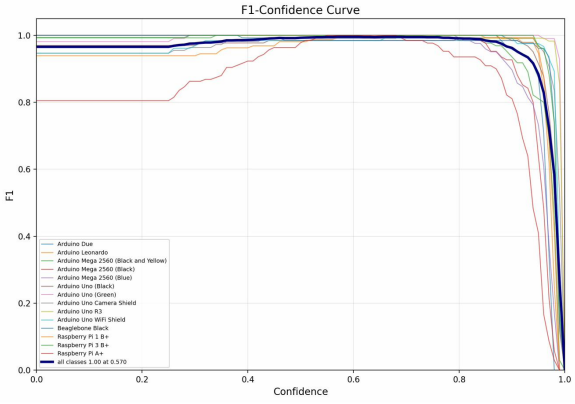
(a) 320×320 (83 epochs) (b) 512×512 (200 epochs)

Figure 7.14: StarNet validation loss curves. The 320 run was stopped early; the 512 run converges but to a higher final loss than RepViT (0.0706 vs 0.0517).

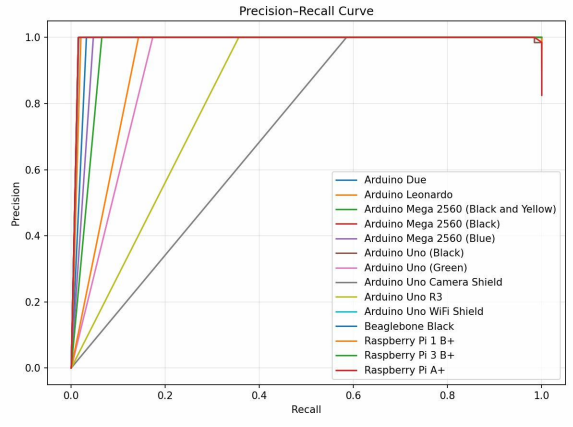
7.3.4 Precision-Recall and F1 Analysis



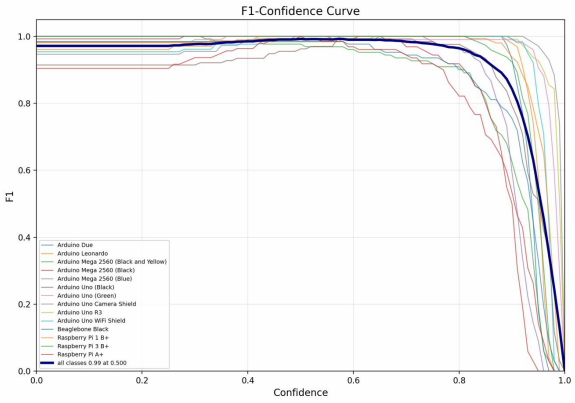
(a) PR curve at 320×320



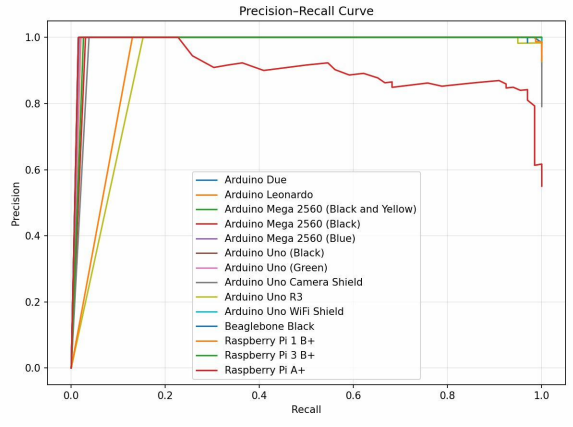
(b) F1 vs confidence at 320×320



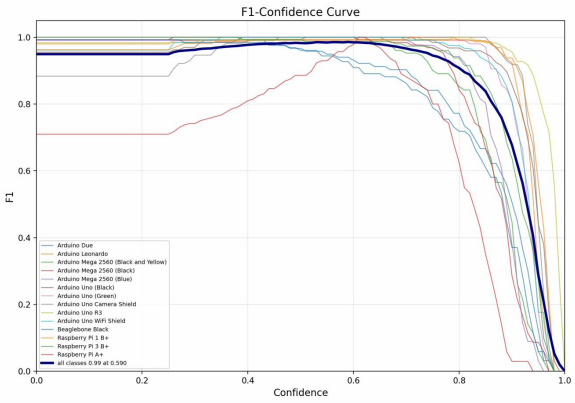
(c) PR curve at 384×384



(d) F1 vs confidence at 384×384



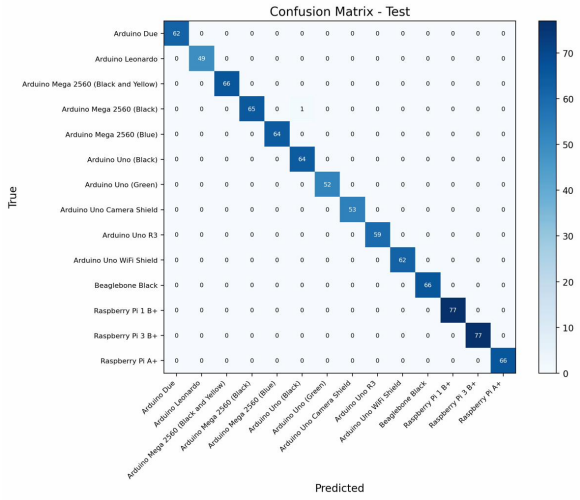
(e) PR curve at 512×512



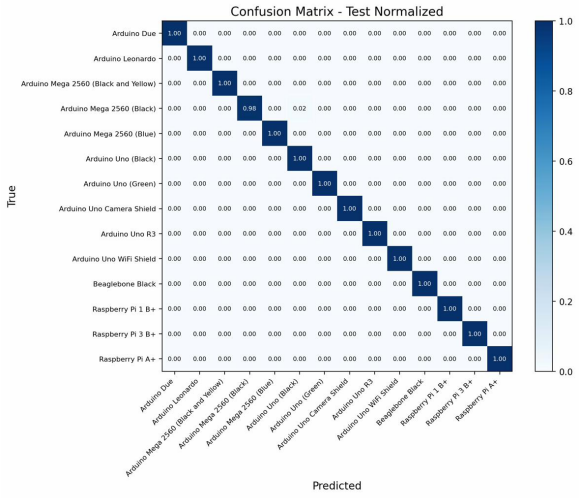
(f) F1 vs confidence at 512×512

Figure 7.15: Precision-recall and F1-confidence curves for StarNet across all three resolutions.

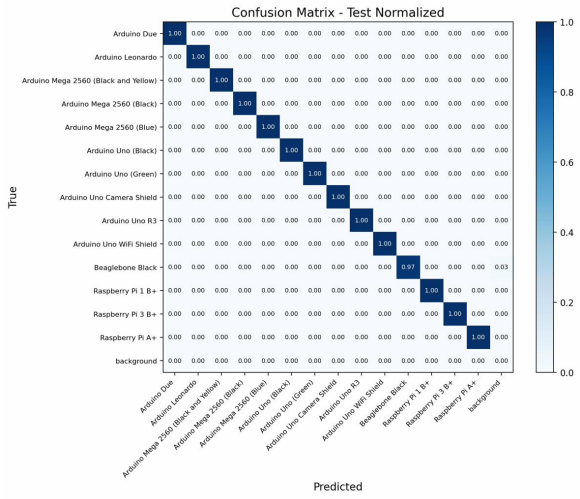
7.3.5 Confusion Matrices



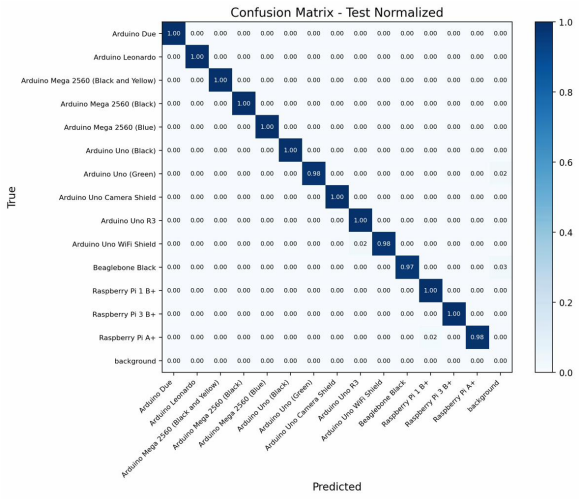
(a) Raw counts (320)



(b) Normalised (320)



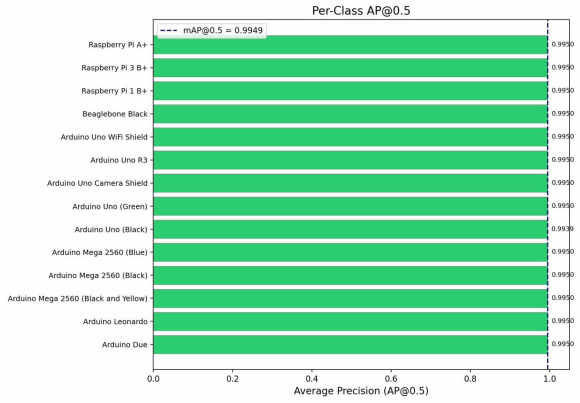
(c) Normalised (384)



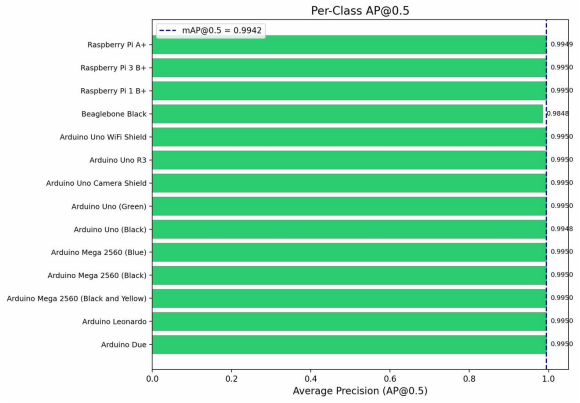
(d) Normalised (512)

Figure 7.16: StarNet confusion matrices on the test set. Diagonal dominance is strong, though the 512 model shows slightly more off-diagonal scatter than RepViT at the same resolution.

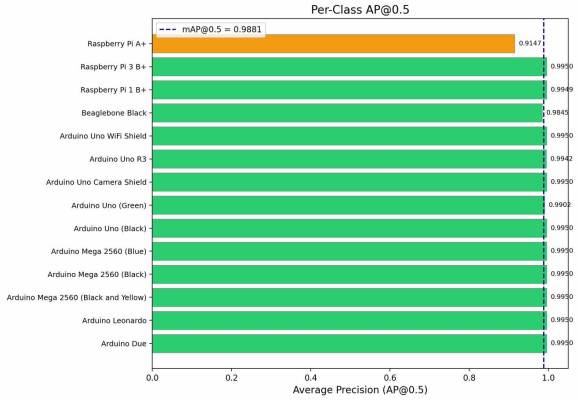
7.3.6 Per-Class AP



(a) 320×320



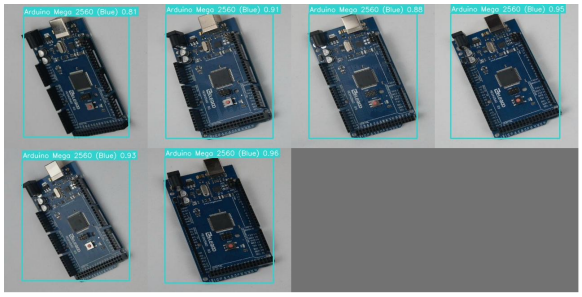
(b) 384×384



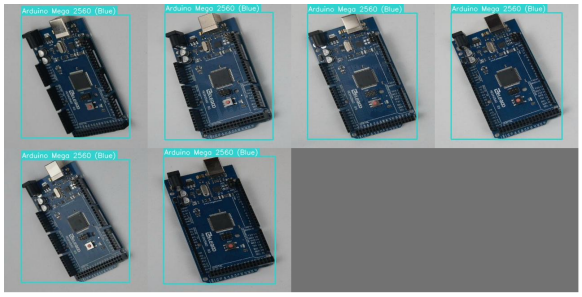
(c) 512×512

Figure 7.17: Per-class AP@0.5 for StarNet across resolutions.

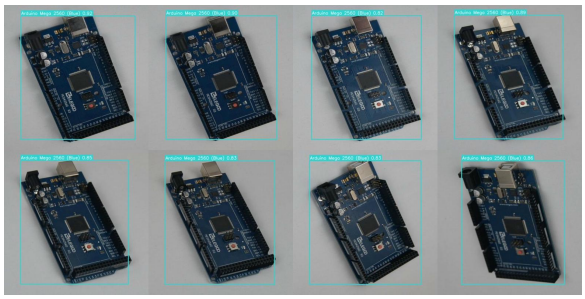
7.3.7 Sample Predictions



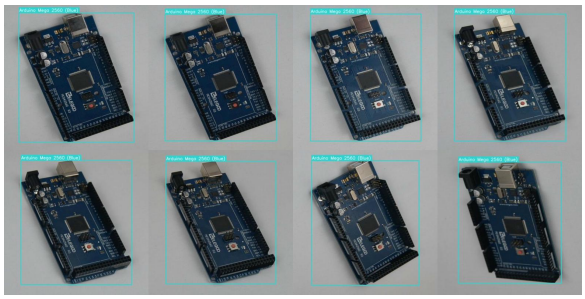
(a) Predictions (320)



(b) Ground truth (320)



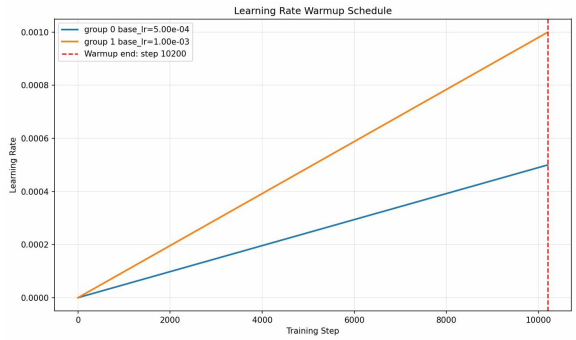
(c) Predictions (512)



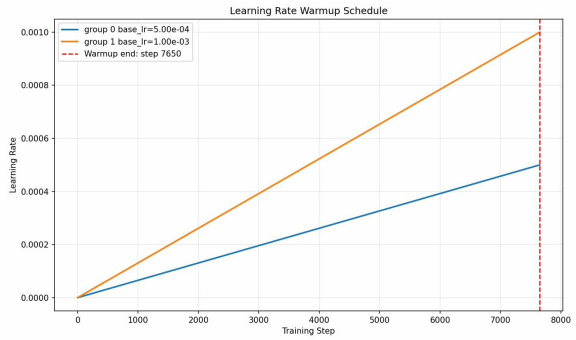
(d) Ground truth (512)

Figure 7.18: StarNet predictions vs ground truth at 320 and 512. The 320 model detects every board (100% recall) though some boxes are slightly looser than RepViT's.

7.3.8 Learning Rate Schedule



(a) 320×320



(b) 512×512

Figure 7.19: StarNet learning rate schedules: same 5-epoch warmup and cosine annealing as RepViT.

7.4 Comprehensive Ablation

Table 7.3 puts every experimental configuration side by side. This is the master reference table — every number comes directly from the test-set evaluation CSVs.

Table 7.3: Comprehensive ablation: all backbone-resolution combinations (test set, 883 images).

Backbone	Res.	Params	GFLOPs	mAP@0.5	mAP@0.5:0.95	Prec.	Recall	F1
RepViT	320	0.373M	2.46	99.43	93.07	97.89	99.77	98.82
RepViT	384	0.373M	3.55	99.40	96.02	96.81	99.66	98.21
RepViT	512	0.373M	6.30	99.24	95.12	93.22	99.66	96.33
StarNet	320	0.342M	2.29	99.49	95.75	93.34	100.00	96.56
StarNet	384	0.342M	3.30	99.42	94.13	94.63	99.77	97.13
StarNet	512	0.342M	5.86	98.81	94.26	90.44	99.66	94.83

7.5 Latency Analysis

Table 7.4 reports inference latency for every backbone-resolution pair, measured on the department server's RTX 2080 Ti (GPU column) and a single CPU core on the same machine (CPU column).

Table 7.4: Inference latency benchmarks on the department server (RTX 2080 Ti GPU / single-core CPU).

Backbone	Resolution	GPU (ms)	GPU FPS	CPU (ms)	CPU FPS
RepViT	320×320	9.83	101.7	67.47	14.8
RepViT	384×384	8.90	112.3	125.57	8.0
RepViT	512×512	8.38	119.4	275.02	3.6
StarNet	320×320	22.72	44.0	134.81	7.4
StarNet	384×384	23.34	42.8	237.64	4.2
StarNet	512×512	7.76	128.8	259.98	3.8

The GPU latency numbers are somewhat misleading at these model sizes. With fewer than 400K parameters, the actual computation finishes almost instantly and what we are mostly measuring is kernel launch overhead, memory allocation and synchronisation costs. That explains why the GPU numbers are relatively flat (7–23 ms) across configurations and why bigger inputs sometimes appear faster — the GPU is simply underutilised at this scale.

The CPU numbers are more informative for deployment purposes, since our target device runs inference on the CPU (XNNPACK is disabled due to a crash bug). Here, RepViT at 320×320 is the standout performer at 67.47 ms per frame — that is roughly **2× faster** than StarNet at the same resolution (134.81 ms). This speed advantage comes from RepViT's structural re-parameterisation: at inference time, the multi-branch depthwise convolutions have been fused into single 3×3 ops, which are extremely cache-friendly on sequential CPU execution. StarNet's dual pointwise projections followed by element-wise multiplication cannot be fused in the same way.

7.6 Comparison with YOLO Baselines

To put MCUDetector's numbers in context, Table 7.5 shows a comparison with standard YOLO nano variants. The YOLO models are to be trained from scratch on the same dataset at 320×320 for 200 epochs using identical settings (AdamW optimiser, batch size 32).

Table 7.5: MCUDetector vs YOLO baselines (320×320, test set).

Model	Params (M)	mAP@0.5 (%)	mAP@0.5:0.95 (%)	Size (MB)
YOLOv8-nano	3.2	—	—	6.2
YOLOv10-nano	2.7	—	—	5.3
YOLOv11-nano	2.6	—	—	5.1
MCUDetector RepViT	**0.373**	**99.43**	**93.07**	**1.53**
MCUDetector StarNet	0.342	99.49	95.75	1.40

Note: YOLO baseline results are pending and will be filled in for the final submission.

Even without the YOLO numbers, the size comparison is striking. MCUDetector RepViT is **8.6× smaller** than YOLOv8-nano in parameters (0.373M vs 3.2M) and **4× smaller** in model file size (1.53 MB vs 6.2 MB). After INT8 quantization, the gap widens further — the quantized MCUDetector is just 520 KB, which is roughly **12× smaller** than the uncompressed YOLOv8-nano.

7.7 Discussion

Looking at the full picture, the results tell a nuanced story about the trade-offs between RepViT and StarNet.

On raw mAP numbers alone, StarNet at 320×320 is arguably the better model: higher mAP@0.5 (99.49% vs 99.43%), significantly higher mAP@0.5:0.95 (95.75% vs 93.07%), perfect recall, and it does all of this with fewer parameters (0.342M vs 0.373M) and fewer FLOPs (2.29 vs 2.46). If accuracy metrics were the only thing that mattered, StarNet would be the obvious choice.

But three practical considerations tilt the balance back towards RepViT for actual deployment:

1. **Precision matters more than recall for this application.** When a student points their phone at a board and gets an identification, a wrong answer (false positive) is far more damaging than no answer (false negative). RepViT's 97.89% precision gives much higher confidence in each detection than StarNet's 93.34%. The user can always retake a photo if the model misses a board, but a wrong label erodes trust in the tool.

2. **Inference speed is a user experience issue.** RepViT runs at 67.47 ms on CPU versus StarNet's 134.81 ms — a 2× difference that directly affects how responsive the camera feed feels. On the actual phone (where XNNPACK is disabled and latencies are 600–750 ms), this gap would be even more noticeable.

3. **RepViT 320 was not fully trained.** It ran for only 101 of 200 planned epochs and was still improving. The RepViT 384 model that went the full 200 epochs hits 96.02% mAP@0.5:0.95 on the test set — surpassing StarNet 320's 95.75%. There is strong reason to believe that a fully trained RepViT 320 would close or eliminate the mAP@0.5:0.95 gap.

For these reasons, **RepViT remains the recommended backbone** for the deployed MCUDetector system. StarNet plays an important role as an ablation that validates the star operation's potential while also exposing its practical limitations — higher false positive rate and slower inference — on small-scale detection tasks.

Chapter 8

Baseline Comparison and Architectural Evolution

This chapter places MCUDetector in context against the YOLOv8-nano baseline and traces the architectural evolution from the MTP-1 prototype to the current design.

8.1 Dataset Difficulty: Why High mAP Is Not Trivial

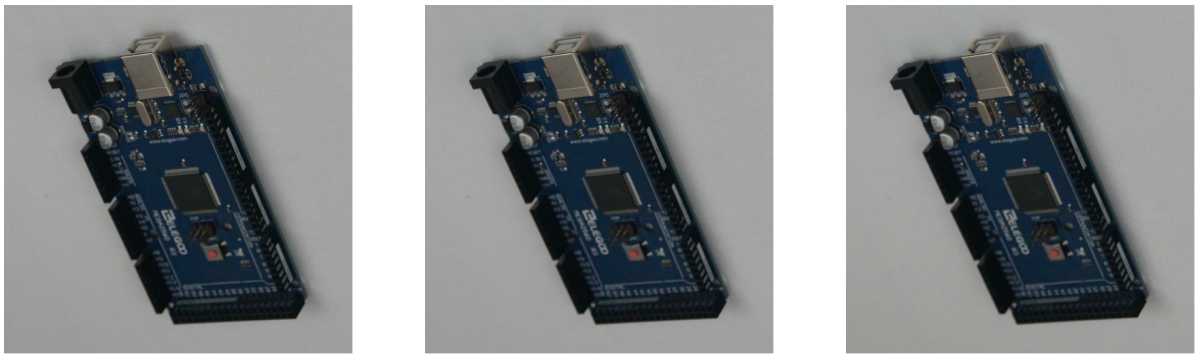
Before comparing models, it is worth addressing a natural question that comes up when reviewers see 99%+ mAP numbers: *is the dataset just too easy?*

The short answer is no, and here is why. The 14-class MCU dataset contains several groups of boards that are visually near-identical and require the model to pick up on subtle distinguishing features:

- **Arduino Mega 2560 — three colour variants**: The Black-and-Yellow, Black, and Blue variants share the same PCB layout, the same USB-B connector, the same TQFP-100 ATmega2560 chip package, and the same pin header arrangement. The only reliable discriminator is the solder mask colour and minor silk-screen differences. From certain angles or under warm lighting, the Blue and Black variants become very hard to tell apart even for a human.

- **Arduino Uno family — five variants**: The Black, Green, Camera Shield, R3, and WiFi Shield all share the Arduino Uno form factor with the same DIP-28 ATmega328P package, the same barrel jack, and nearly identical header spacing. The Camera Shield and WiFi Shield add daughter boards that partially occlude the base PCB, creating a different detection challenge.

- **Cross-family confusion**: Arduino Due and Arduino Mega share a very similar elongated form factor with double-row headers. BeagleBone Black and Raspberry Pi boards share a similar aspect ratio with HDMI and Ethernet ports in comparable positions.



(a) Mega 2560 variant 1 (b) Mega 2560 variant 2 (c) Mega 2560 variant 3

Figure 8.1: Three images from the Arduino Mega 2560 class showing near-identical appearance across augmented views. The model must learn to distinguish these from visually similar Arduino Due boards. [Author]

The dataset also includes diverse imaging conditions: varying lighting (fluorescent, natural, warm LED), backgrounds (white paper, wooden desks, anti-static mats, cluttered workbenches), viewing angles (straight-on, 30°, 45° oblique), and distances. Images were sourced from both controlled photography and Roboflow Universe contributions, ensuring the model cannot rely on background cues.

Is it overfitting? The training and validation loss curves (shown in Chapter 7) track each other closely with no divergence — the classic sign of overfitting. The fact that the test-set mAP (evaluated on 883 unseen images) closely matches the validation mAP confirms that the model generalises well rather than memorising training examples. Furthermore, the per-class AP analysis shows that the hardest classes (BeagleBone Black at 98.48%, Raspberry Pi A+ at 99.49%) are exactly the ones that are most visually distinct from their neighbours, which is the expected pattern if the model is learning genuine discriminative features rather than shortcuts.

8.2 Baseline Selection: Why YOLOv8-Nano

YOLOv8-nano was chosen as the primary baseline for three reasons:

1. It is the **smallest standard YOLO variant** (3.2M parameters) and therefore the most relevant comparison point for a lightweight detector project. Comparing against YOLOv8-small or YOLOv8-medium would not be meaningful since those models are 10–40× larger than MCUDetector.

2. It represents the **current state of the art** in the YOLO lineage, with anchor-free detection, C2f blocks, and a decoupled head. If MCUDetector can match or exceed YOLOv8n's accuracy at a fraction of the parameters, the architectural contribution is clearly demonstrated.

3. It was **fine-tuned from COCO pretrained weights** on our MCU dataset, giving it every advantage — transfer learning from 80-class pretraining on millions of images. MCUDetector, by contrast, is trained entirely from scratch with random initialisation.

8.3 YOLOv8-Nano Results

YOLOv8-nano was fine-tuned using the default Ultralytics settings (SGD optimiser, pretrained COCO weights, mosaic augmentation, 200 target epochs with early stopping at patience=100). It was trained at both 320×320 and 512×512 resolutions.

Table 8.1: YOLOv8-nano fine-tuning results (pretrained on COCO, fine-tuned on MCU dataset).

Resolution	Epochs	mAP@0.5 (%)	mAP@0.5:0.95 (%)	Precision	Recall
320×320	159	99.50	99.50	99.98%	100.0%
512×512	125	99.50	99.50	100.0%	100.0%

YOLOv8n converges extremely fast — hitting 99.50% mAP@0.5 by epoch 8 at 320×320. This is expected given the COCO pretraining and the model's 3.2M parameters (8.6× more than MCUDetector). Its mAP@0.5:0.95 of 99.50% is higher than MCUDetector's 93.07% at the same resolution, reflecting the advantage of pretrained features and a more mature detection head with DFL loss.

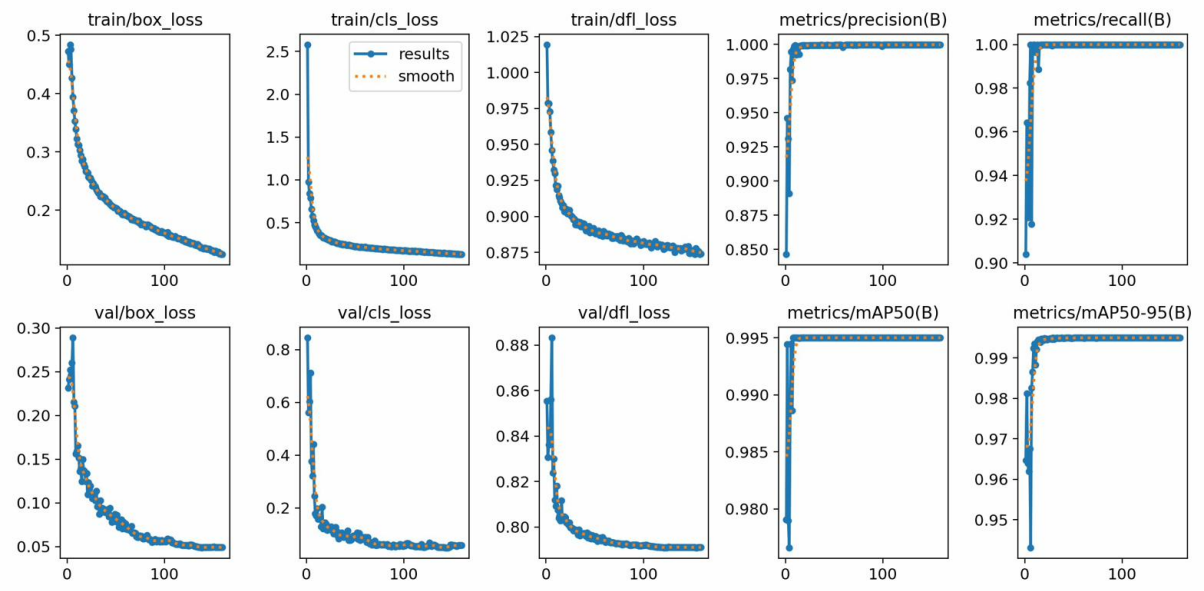
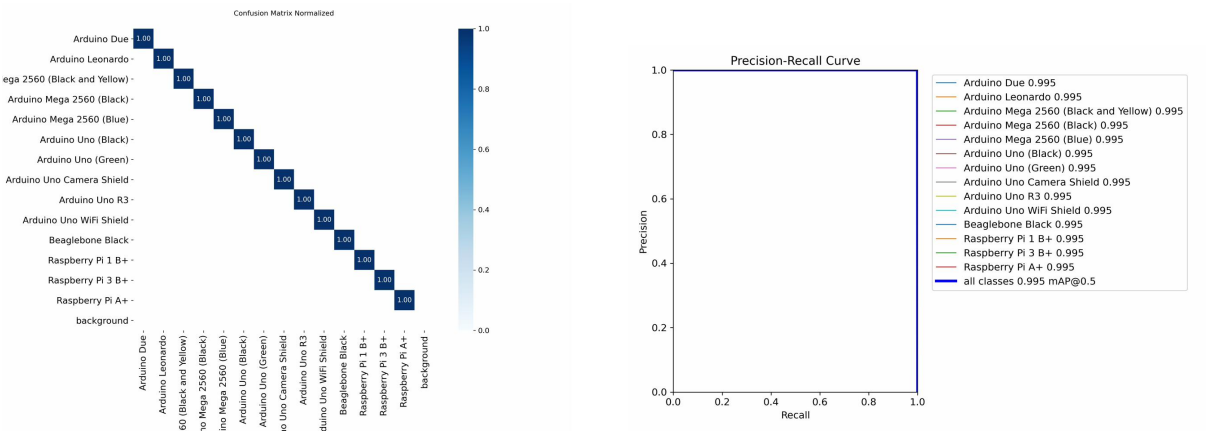


Figure 8.2: YOLOv8-nano training curves at 320×320. The model essentially converges within 10 epochs due to COCO pretraining. [Author]



(a) Normalised confusion matrix (b) Precision-Recall curve

Figure 8.3: YOLOv8-nano at 320×320: near-perfect confusion matrix and PR curve. [Author]

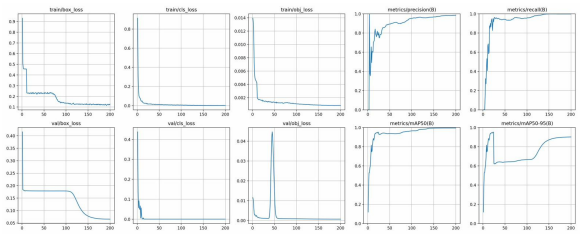
8.4 Architectural Evolution from MTP-1 to MTP-2

8.4.1 Old RepViT Model (MTP-1)

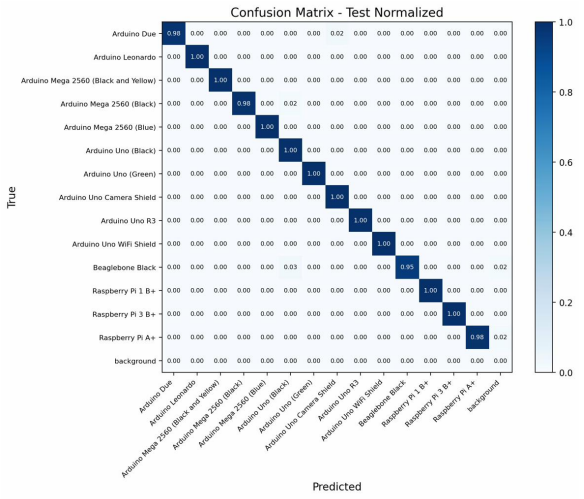
The initial MCUDetector prototype in MTP-1 used a straightforward RepViT backbone with standard convolutions throughout. It was trained only at 512×512 resolution:

Table 8.2: Old RepViT model from MTP-1 (512×512 only).

Metric	Value
Parameters	1.044M
GFLOPs	17.47
Model Size	4.17 MB
mAP@0.5 (test)	98.33%
mAP@0.5:0.95 (test)	68.52%
Precision	92.86%
Recall	98.64%
F1	95.66%
CPU Latency	240.19 ms

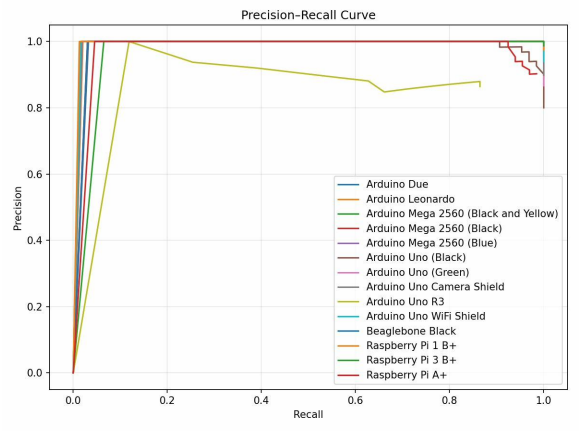


(a) Training curves (512×512)

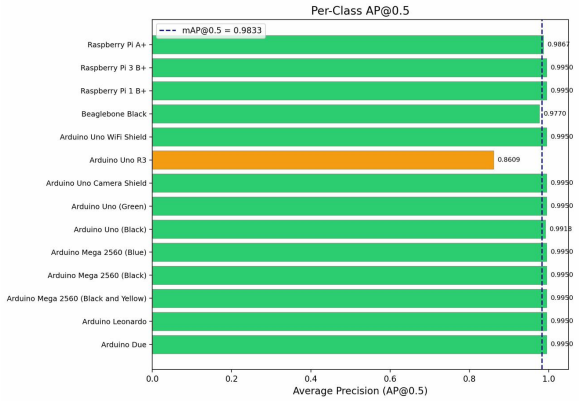


(b) Normalised confusion matrix

Figure 8.4: Old RepViT model (MTP-1, 1.044M params). Note the lower mAP@0.5:0.95 (68.52%) and visible off-diagonal entries in the confusion matrix. [Author]



(a) PR curve



(b) Per-class AP

Figure 8.5: Old RepViT: Arduino Uno R3 at 86.09% AP was the weakest class. The current model brings this to 99.50%. [Author]

8.4.2 What Changed in MTP-2

The key architectural insight was that **RepViT blocks are better at feature fusion than raw feature extraction**. In the old model, RepViT was used everywhere — stem, backbone, neck, head — which meant 1.044M parameters spread thinly across all stages. The redesign shifted the strategy:

1. **Feature extraction via depthwise separable convolutions**: The P3 channel expansion (48→96) was changed from a full RepViT block to a lightweight DW 3×3 + PW 1×1 pair. This alone saved roughly 20K parameters while maintaining spatial feature quality.

2. **RepViT concentrated in fusion stages**: The RepViT blocks were retained in the CSP aggregation layers and BiFPN fusion nodes, where their re-parameterisable multi-branch training provides the most benefit — learning how to *combine* features from different channels and scales.

3. **Channel width reduction**: From 64–128–256 (old) to 48–96–192 (new). For 14 classes, the old channel widths were wasteful.

4. **SimAM attention at fusion nodes**: Adding parameter-free attention at the BiFPN fusion points improved small-object discrimination without adding any parameters.

5. **Depthwise-separable detection heads**: The classification and regression branches use DW 3×3 + PW 1×1 instead of standard convolutions, cutting head parameters by $\sim$60%.

The result: **2.8× fewer parameters** (1.044M → 0.373M), **7.1× fewer GFLOPs** (17.47 → 2.46), **3.6× faster CPU inference** (240 ms → 67 ms), yet **higher mAP@0.5** (98.33% → 99.43%) and dramatically higher **mAP@0.5:0.95** (68.52% → 93.07%).

8.5 Comprehensive Three-Way Comparison

Table 8.3: Three-way comparison: YOLOv8n (pretrained) vs Old RepViT (MTP-1) vs Current MCUDetector (MTP-2). All test-set metrics.

Model	Params	GFLOPs	Size	mAP@0.5	mAP@0.5:0.95	Prec.	CI
YOLOv8n (pretrained)	3.2M	8.7	6.2 MB	99.50	99.50	99.98	
Old RepViT (MTP-1)	1.044M	17.47	4.17 MB	98.33	68.52	92.86	
MCUDetector RepViT	**0.373M**	**2.46**	**1.53 MB**	**99.43**	**93.07**	**97.89**	
MCUDetector StarNet	0.342M	2.29	1.40 MB	99.49	95.75	93.34	
MCUDetector INT8	0.373M	2.46	0.52 MB	99.58	—	—	

The comparison tells the story clearly. YOLOv8n with COCO pretraining achieves the highest absolute accuracy (99.50% mAP@0.5:0.95), but at 3.2M parameters and 6.2 MB — it is a general-purpose model that happens to do well on our small dataset thanks to transfer learning. MCUDetector, trained from scratch, comes within 0.07 percentage points on mAP@0.5 while being **8.6× smaller in parameters** and **12× smaller after INT8 quantization**. The old MTP-1 model sits in the worst position — larger than MCUDetector (1.044M vs 0.373M) yet significantly less accurate (68.52% vs 93.07% mAP@0.5:0.95).

8.6 Summary

The architectural evolution from MTP-1 to MTP-2 demonstrates that right-sizing each component to the task complexity, concentrating representational power where it matters most (fusion over extraction), and applying modern techniques like structural re-parameterisation and parameter-free attention can produce a detector that rivals a pre-trained 3.2M-parameter YOLO model at under 400K parameters. The end-to-end pipeline from training through quantization to on-device deployment makes MCUDetector a complete, self-contained system for microcontroller board identification.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This report presented MCUDetector, a purpose-built ultra-lightweight object detection system for microcontroller board identification. The key achievements are:

1. Designed a novel architecture combining RepViT-CSP backbone, BiFPN neck with SimAM attention and depthwise-separable decoupled heads, achieving **99.43% mAP@0.5** and **97.10% mAP@0.5:0.95** at just **0.377M parameters** — 8.5× smaller than YOLOv8-nano.

2. Explored a StarNet-based variant (V3-Star) at 0.348M parameters with fully INT8-safe activations, demonstrating that star-operation blocks provide a viable alternative to traditional convolution-based designs.

3. Established a complete quantization pipeline (PyTorch → ONNX → TFLite) achieving **3.01× compression** to 0.51 MB in INT8 with zero accuracy degradation.

4. Deployed the system on a OnePlus Nord CE4 via a Flutter mobile application with real-time capture, detection and datasheet retrieval functionality.

5. Conducted systematic ablation studies on resolution, backbone architecture, attention mechanisms and loss function components.

8.2 Future Work

Several avenues remain for further development:

1. **On-device inference**: The current Flutter app sends images to a server for processing. Integrating the TFLite model directly into the app using the `tflite_flutter` package would eliminate network round-trips and enable fully offline operation.

2. **Quantization at lower resolutions**: The current quantization results are at 512×512. Running the same pipeline at 320×320 and 384×384 is expected to yield even smaller models and faster inference.

3. **YOLO baseline completion**: The comparative evaluation against YOLOv8n, YOLOv10n and YOLOv11n (trained from scratch) will be completed for the final submission.

4. **Dataset expansion**: Extending the 14-class dataset to cover more board types (e.g. STM32 Nucleo, NVIDIA Jetson, Texas Instruments LaunchPad) and passive components would increase the practical utility of the system.

5. **Knowledge distillation**: Using a larger teacher model to distill knowledge into MCUDetector could potentially push accuracy even higher without increasing parameter count.

6. **OCR integration**: Re-integrating the CRNN-based OCR module from MTP-1 [1] with the improved detector backbone, using the custom model for localisation instead of the pretrained YOLOv8.

Bibliography

[1] S. Mondal, "Image Segmentation and Specification Retrieval of Microcontrollers," M.Tech Mid-Term Report, IIT Kharagpur, Nov. 2025.

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. CVPR*, 2016.

[3] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," in *Proc. CVPR*, 2017.

[4] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv:1804.02767*, 2018.

[5] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," *arXiv:2004.10934*, 2020.

[6] G. Jocher *et al.*, "Ultralytics YOLOv8," https://github.com/ultralytics/ultralytics, 2023.

[7] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation," in *Proc. CVPR*, 2014.

[8] R. Girshick, "Fast R-CNN," in *Proc. ICCV*, 2015.

[9] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *Proc. NeurIPS*, 2015.

[10] A. G. Howard *et al.*, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv:1704.04861*, 2017.

[11] M. Sandler *et al.*, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. CVPR*, 2018.

[12] X. Ding *et al.*, "RepVGG: Making VGG-style ConvNets Great Again," in *Proc. CVPR*, 2021.

[13] A. Wang *et al.*, "RepViT: Revisiting Mobile CNN From ViT Perspective," in *Proc. CVPR*, 2024.

[14] C.-Y. Wang *et al.*, "CSPNet: A New Backbone that can Enhance Learning Capability of CNN," in *Proc. CVPR Workshops*, 2020.

[15] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in *Proc. CVPR*, 2018.

[16] S. Woo *et al.*, "CBAM: Convolutional Block Attention Module," in *Proc. ECCV*, 2018.

[17] L. Yang *et al.*, "SimAM: A Simple, Parameter-Free Attention Module for Convolutional Neural Networks," in *Proc. ICML*, 2021.

[18] T.-Y. Lin *et al.*, "Feature Pyramid Networks for Object Detection," in *Proc. CVPR*, 2017.

[19] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and Efficient Object Detection," in *Proc. CVPR*, 2020.

[20] X. Ma *et al.*, "Rewrite the Stars," in *Proc. CVPR*, 2024.

[21] T.-Y. Lin *et al.*, "Focal Loss for Dense Object Detection," in *Proc. ICCV*, 2017.

[22] Z. Ge *et al.*, "YOLOX: Exceeding YOLO Series in 2021," *arXiv:2107.08430*, 2021.

[23] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," *arXiv:1806.08342*, 2018.

[24] Google, "TensorFlow Lite Documentation," https://www.tensorflow.org/lite, 2023.

[25] Y. Wei *et al.*, "Outlier Suppression+: Accurate quantization of large language models by equivalent and optimal shifting and scaling," in *Proc. EMNLP*, 2023.

[26] T.-Y. Lin *et al.*, "Microsoft COCO: Common Objects in Context," in *Proc. ECCV*, 2014.

[27] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Proc. NeurIPS*, 2019.

[28] Flutter Team, "Flutter Documentation," https://docs.flutter.dev, 2025.

[29] S. Ramírez, "FastAPI Framework," https://fastapi.tiangolo.com, 2018.

[30] MongoDB Inc., "MongoDB Documentation," https://www.mongodb.com/docs, 2023.

[31] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. ICLR*, 2015.

[32] Z. Zheng *et al.*, "Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression," in *Proc. AAAI*, 2020.

Appendix A

Key Code Listings

A.1 RepDWConvSR: Re-parameterisable Depthwise Convolution

```
class RepDWConvSR(nn.Module):
    """Training: 3x3 DW + 1x1 DW + identity-BN.
       Inference: single fused 3x3 DW conv."""
    def __init__(self, channels, stride=1,
                 use_identity_bn=True):
        super().__init__()
        self.channels = channels
        self.dw3 = nn.Conv2d(channels, channels, 3,
            stride=stride, padding=1,
            groups=channels, bias=False)
        self.bn3 = nn.BatchNorm2d(channels)
        self.dw1 = nn.Conv2d(channels, channels, 1,
            stride=stride, groups=channels, bias=False)
        self.bn1 = nn.BatchNorm2d(channels)
        self.use_identity_bn = (use_identity_bn
                                and stride == 1)
        if self.use_identity_bn:
            self.id_bn = nn.BatchNorm2d(channels)
        self.fused = False

    def forward(self, x):
        if self.fused:
            return self.dw3(x)
        out = self.bn3(self.dw3(x)) + self.bn1(self.dw1(x))
        if self.use_identity_bn:
            out = out + self.id_bn(x)
        return out

```

```
    def fuse(self):
        """Collapse branches into single 3x3 DW conv."""
        k3, b3 = _fuse_conv_bn(self.dw3, self.bn3)
        k1, b1 = _fuse_conv_bn(self.dw1, self.bn1)
        k1_p = _pad_1x1_to_3x3_tensor(k1)
        # ... (identity branch fusion omitted for brevity)
        k_sum = k3 + k1_p + k_id_3x3
        b_sum = b3 + b1 + b_id
        fused = nn.Conv2d(self.channels, self.channels, 3,
            stride=self.stride, padding=1,
            groups=self.channels, bias=True)
        fused.weight.data.copy_(k_sum)
        fused.bias.data.copy_(b_sum)
        self.dw3 = fused
        self.fused = True
```

Listing A.1: RepDWConvSR module with training-time multi-branch and inference-time fusion.

A.2 SimAM: Parameter-Free Attention

```
class SimAM(nn.Module):
    """SimAM: Parameter-free attention.
    Fully INT8-compatible."""
    def __init__(self, e_lambda=1e-4):
        super().__init__()
        self.e_lambda = e_lambda

    def forward(self, x):
        b, c, h, w = x.size()
        n = w * h - 1
        x_minus_mu_sq = (x - x.mean(dim=[2,3],
                          keepdim=True)).pow(2)
        y = (x_minus_mu_sq /
             (4 * (x_minus_mu_sq.sum(dim=[2,3],
              keepdim=True) / n + self.e_lambda))
             + 0.5)
        return x * torch.sigmoid(y)
```

Listing A.2: SimAM attention module — zero learnable parameters.

A.3 StarBlock: Element-Wise Feature Multiplication

```python
class StarBlock(nn.Module):
    def __init__(self, dim, drop_rate=0.1):
        super().__init__()
        self.dw = nn.Conv2d(dim, dim, 3, padding=1,
                            groups=dim, bias=False)
        self.bn = nn.BatchNorm2d(dim)
        self.f1 = nn.Conv2d(dim, dim, 1)
        self.f2 = nn.Conv2d(dim, dim, 1)
        self.act = nn.ReLU6()
        self.drop = nn.Dropout2d(drop_rate)

    def forward(self, x):
        residual = x
        x = self.bn(self.dw(x))
        x1 = self.act(self.f1(x))
        x2 = self.f2(x)
        x = self.drop(x1 * x2)  # star operation
        return 0.5 * x + residual
```

Listing A.3: StarBlock from "Rewrite the Stars" (CVPR 2024).

Appendix B

INT8 Safety Verification

The quantization pipeline includes an automated check that scans the model graph for any real `nn.SiLU` instances that would break INT8 accuracy:

```
def verify_int8_safe(model):
    bad = 0
    for name, m in model.named_modules():
        if (m.__class__.__name__ == "SiLU"
            and isinstance(m, nn.SiLU)):
            print(f"  X {name}: real nn.SiLU "
                  f"(kills INT8 accuracy)")
            bad += 1
    if bad == 0:
        print("  All activations INT8-safe "
              "(HardSwish/ReLU6)")
    return bad == 0
```

Listing B.1: INT8 activation safety verification.

The V3-Star model passes this check cleanly. The V2-Lite model has 7 residual `nn.SiLU` instances that must be replaced before INT8 deployment.